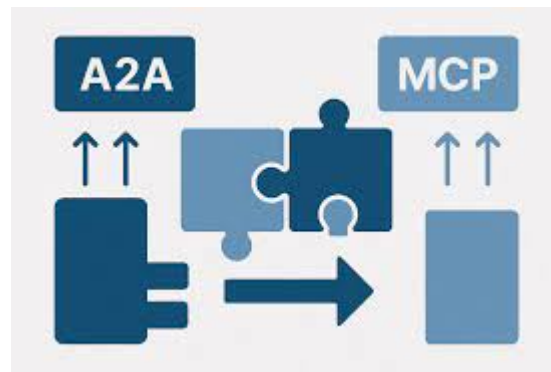




By JP

Agent2Agent (A2A) Protocol is an open standard designed to enable seamless communication and ***collaboration*** between AI agents.

In a world where agents are built using diverse frameworks and by different vendors, A2A provides a common language, breaking down silos and fostering interoperability.

What is A2A?

Agent2Agent (A2A) Protocol is an open standard designed to solve a fundamental challenge in the rapidly evolving landscape of artificial intelligence:

How do AI agents, built by different teams, using different technologies, and owned by different organizations, communicate and collaborate effectively?

Imagine Sarah, who wants to plan an international trip. She asks her primary AI assistant:

"Plan me a 7-day vacation in Italy."

The assistant immediately reaches out to specialized agents:

- **Flight Agent** → Finds and books flights.
- **Hotel Agent** → Reserves the best accommodation.
- **Tour Agent** → Suggests and books local experiences.
- **Currency & Advisory Agent** → Provides exchange rates and safety updates.

From Sarah's perspective, everything feels smooth and effortless—she gets a complete trip plan in minutes.

But behind the scenes, without a **common communication protocol**, each of these agents would require one-off, custom integrations. As more agents are added, the system becomes tangled, hard to maintain, and nearly impossible to scale.

With standardized communication, however, these agents can cooperate naturally—making Sarah's trip (and the developer's job) much simpler.

A2A Solution

A2A provides a standardized way for these independent, often "opaque" (black-box) agentic systems to interact. It defines:

- **A common transport and format:** JSON-RPC 2.0 over HTTP(S) for how messages are structured and transmitted.
- **Discovery mechanisms (Agent Cards):** How agents can advertise their capabilities and be found by other agents.
- **Task management workflows:** How collaborative tasks are initiated, progressed, and completed. This includes support for tasks that may be long-running or require multiple turns of interaction.
- **Support for various data modalities:** How agents exchange not just text, but also files, structured data (like forms), and potentially other rich media.
- **Core principles for security and asynchronicity:** Guidelines for secure communication and handling tasks that might take significant time or involve human-in-the-loop processes.

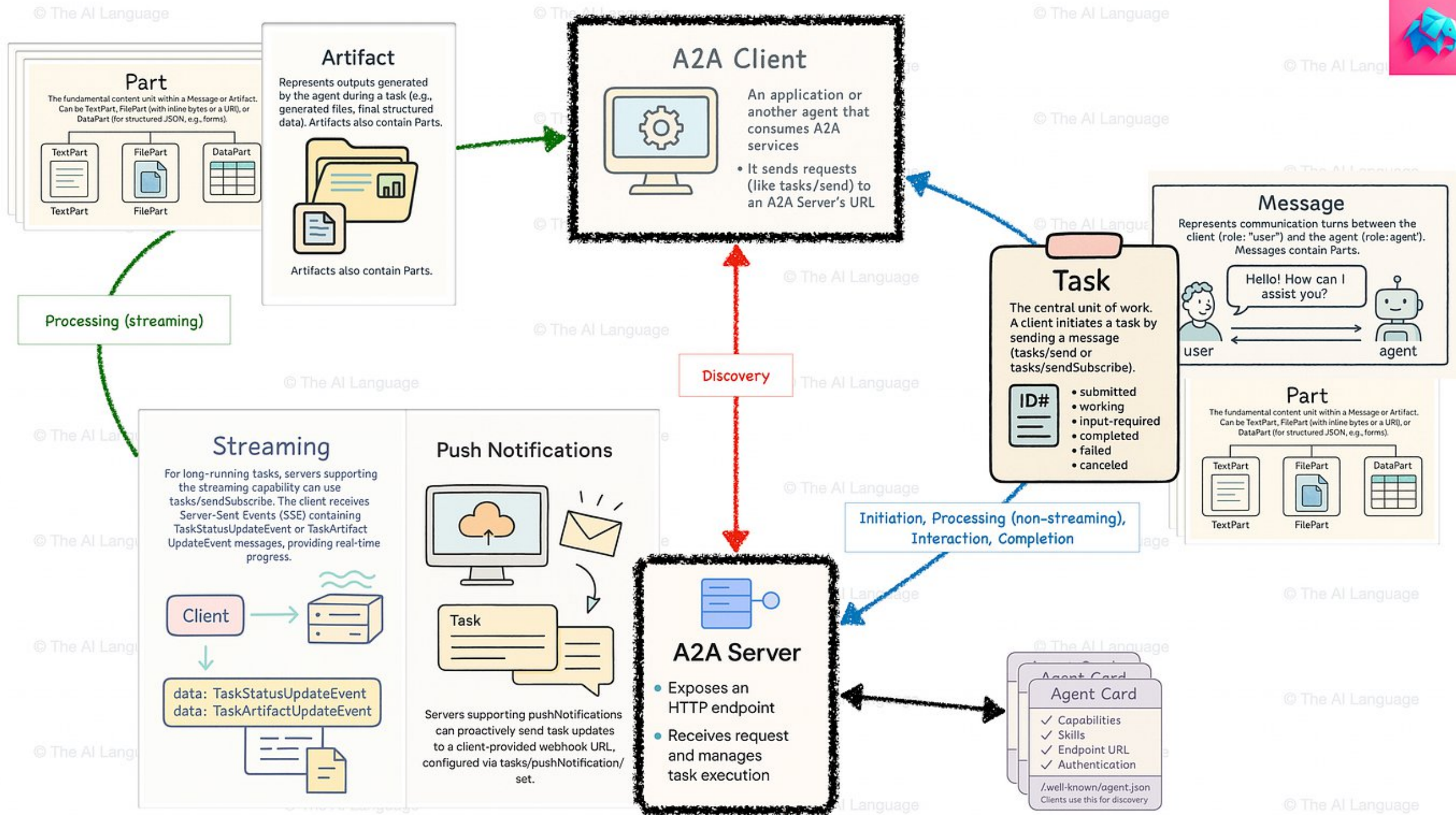
🔑 Key Design Principles of A2A

- **Simplicity** – Build on proven standards (HTTP, JSON-RPC, SSE) instead of creating new protocols.
- **Enterprise Readiness** – Incorporate authentication, security, privacy, tracing, and monitoring from the start.
- **Asynchronous First** – Support long-running tasks and intermittent connectivity with streaming and push notifications.
- **Modality Agnostic** – Enable communication across diverse content types, not just text.
- **Opaque Execution** – Allow agents to collaborate through declared capabilities without revealing internal logic, tools, or memory—protecting IP and ensuring security.

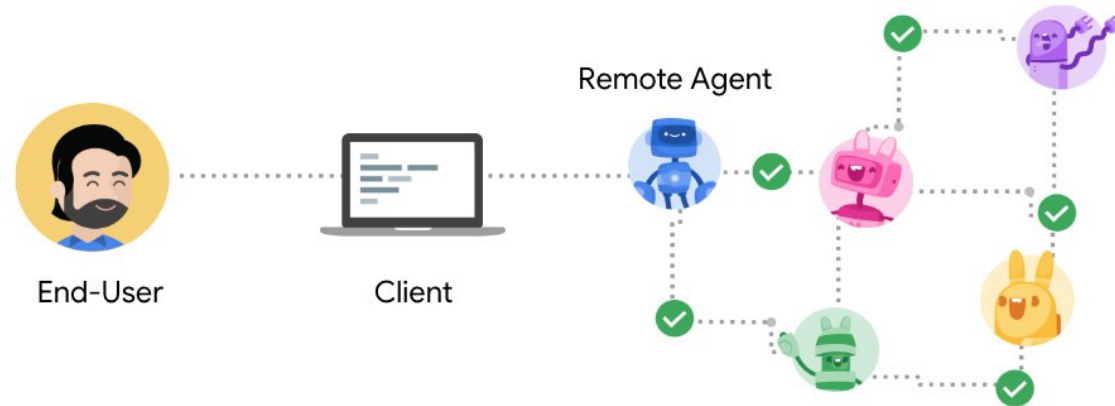
🌟 Benefits of Using A2A

- **Works Across Systems** – Agents from different vendors and platforms can easily talk to each other.
- **Smarter Together** – Combine the strengths of specialized agents to build richer applications.
- **Less Hassle** – A common standard means fewer custom integrations and more focus on real value.
- **Boosts Innovation** – Makes it easier to create and plug in new specialized agents.
- **Ready for the Future** – Flexible enough to grow as AI technology evolves.

👉 In short, A2A creates a **common language for agents**, unlocking more collaboration, power, and real-world impact.



Key Concepts in A2A



Core Actors

- **User:** The end user (human or automated service) who initiates a request or goal that requires agent assistance.
- **A2A Client (Client Agent):** An application, service, or another AI agent that acts on behalf of the user to request actions or information from a remote agent. The client initiates communication using the A2A protocol.
- **A2A Server (Remote Agent):** An AI agent or agentic system that exposes an HTTP endpoint implementing the A2A protocol. It receives requests from clients, processes tasks, and returns results or status updates. The remote agent operates as an "opaque" system from the client's perspective, meaning the client doesn't need to know its internal workings, memory, or tools.

Fundamental Communication Elements

Fundamental Communication Elements

1. Agent Card

- A JSON file (e.g., `/.well-known/agent-card.json`) that describes an A2A server.
- Includes: agent name, description, service endpoint, version, supported features (streaming, push), skills, input/output types, and authentication.
- Used by clients to **discover agents** and learn how to interact with them securely.

2. Task

- Represents work an agent carries out (e.g., "book a flight," "generate a report").
- Each task has a **unique ID** and follows a lifecycle: submitted → working → input-required → completed/failed.
- Tasks are **stateful** and may involve multiple exchanges.

3. Message

- A single communication turn between client ↔ agent.
- Has a **role**:
 - `"user"` → client-sent
 - `"agent"` → server-sent
- Contains one or more **Parts** (the actual content).
- Each has a **messageId** for unique tracking.

4. Part

- The **smallest content unit** inside a Message or Artifact.
- Types:
 - **TextPart** → plain text
 - **FilePart** → files (inline or by URI, with metadata)
 - **DataPart** → structured JSON data (forms, parameters, machine-readable info)

5. Artifact

- A **tangible output/result** of a task.
- Examples: documents, images, spreadsheets, structured data.
- Built from one or more **Parts** and can be **streamed incrementally**.
- Returned when tasks reach **completed** state.

Interaction Mechanisms

1. Request / Response (Polling)

- Client sends a request → server replies.
- For long tasks, server may return “working,” and client keeps checking until done.

2. Streaming (SSE – Server-Sent Events)

- For live updates or incremental results.
- Client opens a stream → server sends continuous updates (status, messages, artifacts).

3. Push Notifications

- For very long tasks where keeping a connection is impractical.
- Client gives a webhook → server sends status updates (e.g., completed, failed, needs input).

 In simple terms:

- **Polling** = “Keep asking, are you done yet?”
- **Streaming** = “Stay connected, I’ll update you live.”
- **Push** = “I’ll call you when it’s ready.”

📌 Other Important Concepts

- **Context (contextId)** – Groups related tasks under one identifier for continuity across interactions.
 - **Transport & Format** – Uses HTTP(S) with JSON-RPC 2.0 for requests and responses.
 - **Authentication & Authorization** – Based on standard web security (OAuth, API keys). Requirements are declared in the **Agent Card**.
 - **Agent Discovery** – Clients find **Agent Cards** to learn about available agents and their capabilities.
 - **Extensions** – Agents can declare **custom protocol extensions** in their Agent Card.
-

👉 Together, these concepts ensure A2A is **secure, interoperable, and flexible** for building collaborative AI systems.

A2A and MCP: Complementary Protocols for Agentic Systems

💛 A2A ❤️ MCP

- Two main protocol types are shaping AI interoperability:
 1. **Tool/Resource Connections** → handled by **MCP (Model Context Protocol)**.
 2. **Agent-to-Agent Collaboration** → handled by **A2A (Agent2Agent Protocol)**.
- **MCP**: Best for letting agents connect to external tools, APIs, and resources.
- **A2A**: Best for enabling agents to talk, collaborate, and coordinate with each other.
- **Key Insight:**
 - 👉 Agentic applications need **both**:
 - **MCP for tools**
 - **A2A for agents**

🔑 Why Different Protocols?

🔧 Tools & Resources

- Well-defined inputs/outputs, predictable behavior.
- Perform **specific, stateless functions** (e.g., calculator, database query, weather API).
- Mostly **single request-response** interactions.

🤖 Agents

- Autonomous systems that **reason, plan, and collaborate**.
- Can use multiple tools and maintain **state across longer interactions**.
- Behavior may be emergent, less predictable, and involve **multi-turn dialogues**.

👉 Key Takeaway

- **Agents use tools** to get info and act.
- **Agents collaborate with agents** to solve broader, more complex tasks.
- That's why we need **MCP for tools** and **A2A for agents**.

🔧 Model Context Protocol (MCP)

- **Focus** → Standardizes how AI models/agents connect to **tools, APIs, and data sources**.
- **Mechanism** → Defines a structured way to:
 - Describe tool capabilities (like LLM function calling).
 - Send inputs → receive structured outputs.
- **Use Cases:**
 - LLM calling an external API (e.g., stock prices).
 - Agent querying a database with parameters.
 - Connecting to predefined functions/services.
- **Ecosystem Goal** → Build a shared ecosystem where **tool providers expose services easily** and **agents consume them seamlessly**.

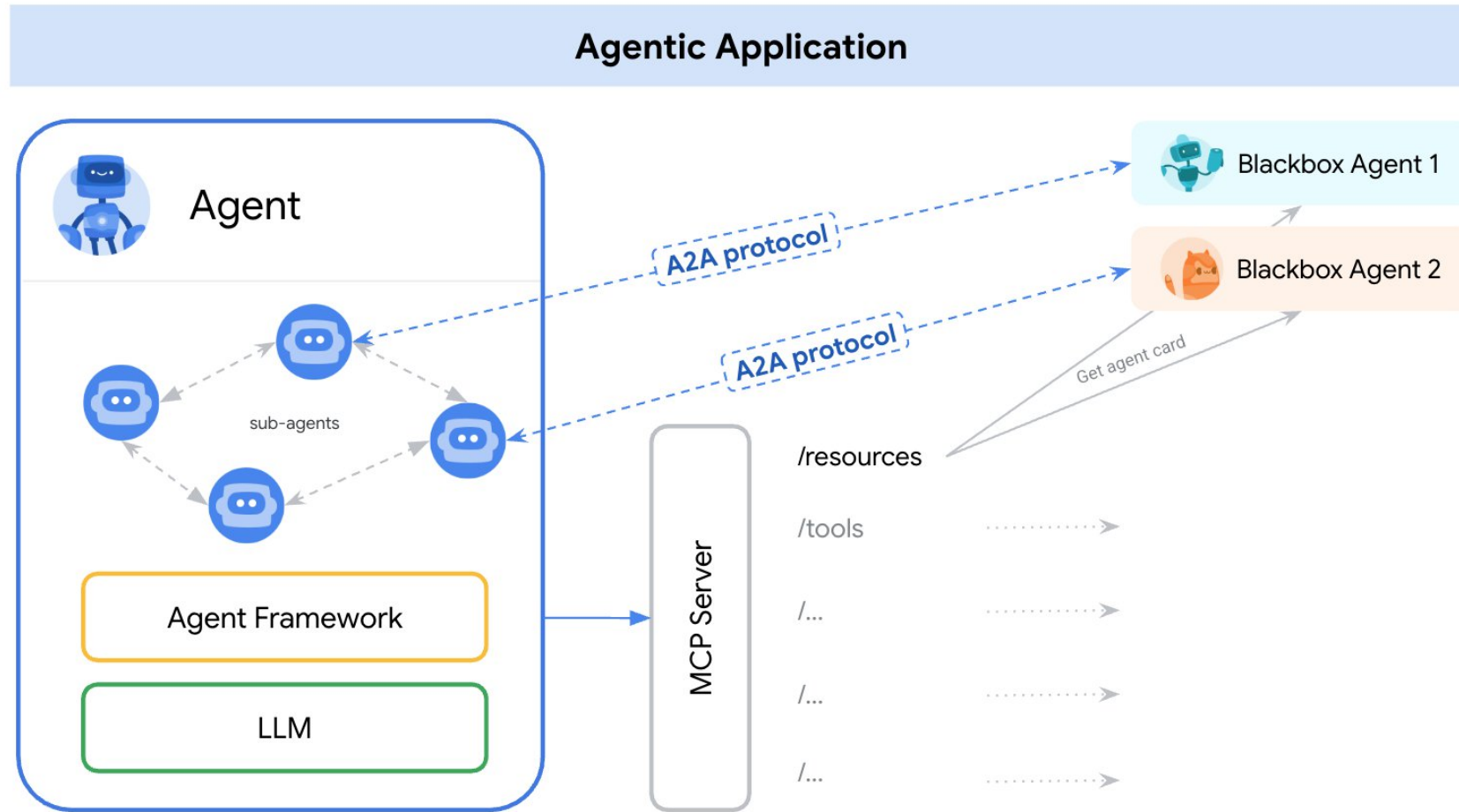
👉 Agent2Agent Protocol (A2A)

- **Focus** → Standardizes how independent AI agents **communicate and collaborate as peers**.
- **Mechanism:**
 - Discover each other's **skills & capabilities** (Agent Cards).
 - Negotiate **interaction modes** (text, files, structured data).
 - Manage **shared, stateful, long-running tasks**.
 - Exchange **context, instructions, and multi-part results**.
- **Use Cases:**
 - Customer service agent → delegates billing inquiry to billing agent.
 - Travel planner → coordinates with flight, hotel, and activity agents.
 - Agents collaborating on evolving projects.
- **Key Difference vs MCP:**
 - MCP = calling tools/functions (stateless, transactional).
 - A2A = **dynamic, stateful, multi-modal agent-to-agent collaboration**.

MCP vs A2A - Comparison Table

Aspect	MCP (Model Context Protocol)	A2A (Agent2Agent Protocol)
Focus	Connects agents/models to tools, APIs, data sources	Peer-to-peer communication & collaboration between agents
Mechanism	Structured tool descriptions, inputs → outputs (like LLM function calling)	Agent Cards, negotiate modalities, manage stateful & long-running tasks
Use Cases	API calls, database queries, connect to predefined services	Customer support delegation, travel planning, collaborative projects
Key Difference	Transactional, stateless tool calls	Dynamic, stateful, multi-modal agent collaboration

How A2A and MCP Complement Each Other



An agentic application might use A2A to communicate with other agents, while each agent internally uses MCP to interact with its specific tools and resources.

Example Scenario: The Auto Repair Shop

1. Customer Interaction (User ↔ Shop Manager via A2A)

- Customer: "My car is making a rattling noise."
- Shop Manager agent asks follow-up questions:
 - "Can you send a video of the noise?"
 - "I see some fluid leaking. How long has this been happening?"
- → **A2A enables multi-turn, conversational diagnosis.**

2. Internal Tool Usage (Mechanic Agent ↔ Tools via MCP)

- Mechanic agent uses tools through **MCP** calls:
 - `scan_vehicle_for_error_codes(vehicle_id="XYZ123")`
 - `get_repair_procedure(error_code="P0300", make="Toyota", model="Camry")`
 - `raise_platform(height_meters=2)`
- → **MCP provides structured, predictable tool access.**

3. Supplier Interaction (Mechanic Agent ↔ Parts Supplier via A2A)

- Mechanic agent: "Do you have part #12345 for Toyota Camry 2018?"
- Supplier agent responds with stock info and possible ordering.
- → **A2A connects agents for negotiation and coordination.**

Key Takeaway

- **A2A** → Conversational, task-oriented interactions (customer ↔ shop manager, shop ↔ supplier).
- **MCP** → Structured, tool-oriented interactions (mechanic ↔ diagnostic tools, manuals, lifts).

Agent Discovery in A2A

Why it matters:

- For agents to collaborate, they must **find each other** and understand what the other can do.
- A2A uses the **Agent Card** as the standard way to describe an agent.

Agent Card = Digital Business Card

A JSON document that provides key details about an agent:

- **Identity** → Name, description, provider info.
- **Service Endpoint** → URL to reach the A2A service.
- **Capabilities** → Supported features (e.g., streaming, push notifications).
- **Authentication** → Schemes required (e.g., Bearer token, OAuth2).
- **Skills** → Tasks the agent can perform (id, name, description, input/output modes, examples).

How it's used:

- Client agents parse the Agent Card to check:
 -  Is this the right agent for the task?
 -  What skills does it offer?
 -  How should requests be structured?
 -  How to communicate securely?

 In short: **Agent Cards** let agents find, identify, and securely interact with each other in A2A.

Discovery Strategy: Well-Known URI

Mechanism

- Agent hosts its **Agent Card** at a standardized path:

```
pgsql
```

```
https://{agent-domain}/.well-known/agent-card.json
```

- Based on RFC 8615 for well-known URIs.

Process

1. Client knows or discovers the agent's domain (e.g., `smart-thermostat.example.com`).
2. Client sends HTTP GET →
`https://smart-thermostat.example.com/.well-known/agent-card.json`.
3. Server returns the **Agent Card JSON** if available.

Advantages

- Simple and standardized.
- Works with crawlers or automated systems.
- Reduces discovery to just **knowing the agent's domain**.

Considerations

- Best for **public agents** or within an org that owns the domain.
- Endpoint can require authentication if sensitive.

Discovery Strategy: Curated Registries

Mechanism

- A central **registry/catalog** stores Agent Cards.
- Clients query it to find agents by skills, tags, provider, or capabilities.

Process

1. Agent servers (or admins) register their **Agent Cards** with the registry.
2. Client queries registry (e.g., *"find image-generation agents with streaming"*).
3. Registry returns matching **Agent Cards**.

Advantages

- Centralized **management & governance**.
- Search based on **capabilities** (not just domains).
- Can enforce **policies, access controls, trust**.
- Works for **enterprise/team catalogs** or **public marketplaces**.

Considerations

- Needs a **separate registry service**.
- A2A doesn't yet define a standard API for registries (future potential).

Discovery Strategy: Direct Configuration / Private Discovery

Mechanism

- Client is **pre-configured** with Agent Card details or a URL.
- Can be from:
 - Hardcoded values
 - Local config file
 - Environment variable
 - Proprietary/private API endpoint

Process

- Depends on the application's setup and deployment strategy.

Advantages

- Very **simple & reliable** for static/known agent relationships.
- Useful for **private agents** or **development/testing**.

Considerations

- Not flexible for **dynamic discovery**.
- Any changes to the Agent Card may require client **reconfiguration**.
- Proprietary discovery APIs are **not standardized** in A2A.

Securing Agent Cards

Sensitive info that may appear in Agent Cards

- Internal/restricted **service URLs**.
- Non-secret auth details (e.g., OAuth token URL).
- Descriptions of **sensitive or internal skills**.
-  Never store **plaintext secrets** in Agent Cards.

Protection Mechanisms

1. **Access Control** → Secure the HTTP endpoint (e.g., `/.well-known/agent-card.json`).
2. **mTLS** → Use **mutual TLS** for strong client authentication.
3. **Network Restrictions** → Restrict access to **specific IPs, VPCs, or private networks**.
4. **Authentication** → Require OAuth2 tokens, API keys, or other **standard HTTP auth** to fetch the card.
5. **Selective Disclosure** → Registries return **different levels of detail** based on client's identity & permissions (e.g., public vs partner access).

Best Practice

- Treat Agent Cards like **sensitive metadata**.
- Use **dynamic, out-of-band credentials** instead of static secrets in the card.

Enterprise-Ready Features for A2A

- **Built for Enterprises** → Uses existing, proven **HTTP-based standards** instead of proprietary protocols.
- **Seamless Integration** → Works with current **security, monitoring, governance, and identity systems**.
- **No Reinventing the Wheel** → Leverages familiar web practices (auth, logging, tracing, auditing).
- **Opaque Agents** → Agents don't expose internal memory, tools, or resources — aligns with **client/server security models**.
- **Enterprise Benefits** → Reuse existing infrastructure → easier adoption, stronger security, smoother operations.

1. Transport Level Security (TLS)

- **HTTPS Required** → All A2A communication in production must use **HTTPS**.
 - **Modern TLS** → Use **TLS 1.2+** with strong, industry-standard cipher suites.
 - **Server Verification** → Clients must **validate TLS certificates** against trusted CAs to confirm server identity and prevent MITM attacks.
-  Ensures **confidentiality** (no eavesdropping) and **integrity** (no tampering) of data in transit.

🔑 2. Authentication

- **Standard Web Mechanisms** → Uses **OAuth2**, **OpenID Connect**, **API keys**, etc. (advertised in the Agent Card).
- **Transport-Level Identity** → JSON-RPC payloads don't carry identity; authentication happens via **HTTP headers**.
- **Agent Card Declaration** → Server lists supported auth schemes (`apiKey` , `http` , `oauth2` , `openIdConnect`) in its **security** section.
- **Credential Handling** → Clients obtain tokens/keys **out-of-band** (e.g., OAuth flow, secure key distribution).
- **HTTP Header Transmission** → Credentials passed via headers:
 - `Authorization: Bearer <token>`
 - `API-Key: <key_value>`
- **Server Validation** → Server validates credentials on each request.
 - If invalid/missing → returns **401 Unauthorized** or **403 Forbidden**.
 - 401 responses SHOULD include **WWW-Authenticate** header to guide the client.
- **In-Task Authentication (Secondary Credentials)** →
 - If a task needs additional credentials (e.g., for external tools), the server marks the task **input-required** and requests them.
 - Client fetches new credentials out-of-band and provides them back (if proxying) or uses them directly.

✅ Ensures **secure, flexible, and enterprise-aligned authentication** for agents.

✓ 3. Authorization

- **Post-Authentication Step** → After verifying identity, the A2A Server decides **what the client is allowed to do**.
- **Granular Control** → Based on authenticated identity (user, client app, or both).
- **Skill-Based Authorization** →
 - Permissions tied to **skills** advertised in the Agent Card.
 - Example: OAuth scopes map to which skills can be invoked.
- **Data & Action-Level Control** →
 - Agents must enforce authorization before accessing sensitive **data** or performing sensitive **actions**.
 - Agent acts as a **gatekeeper** to backend systems and tools.
- **Principle of Least Privilege** →
 - Only grant the **minimum permissions** needed for the client's tasks.

✓ Ensures secure, fine-grained access control aligned with enterprise policies.

4. Data Privacy & Confidentiality

- **Sensitivity Awareness** → Treat all exchanged **Messages** and **Artifacts** as potentially sensitive.
 - **Regulatory Compliance** → Follow applicable laws (e.g., **GDPR**, **CCPA**, **HIPAA**) based on domain/data.
 - **Data Minimization** → Only exchange the **minimum necessary information**.
 - **Secure Handling** →
 - In transit → protected with **TLS**.
 - At rest → secured per enterprise policies & regulations.
- ✅ Ensures A2A communications respect **privacy, compliance, and enterprise security standards**.

5. Tracing, Observability & Monitoring

- **Distributed Tracing**
 - Use systems like **OpenTelemetry, Jaeger, Zipkin**.
 - Propagate trace context (e.g., `traceparent` , `tracestate` headers).
 - Enables **end-to-end visibility** for debugging & performance analysis.
 - **Comprehensive Logging**
 - Log details on both client & server sides.
 - Include identifiers: `taskId` , `sessionId` , correlation IDs, trace context.
 - **Metrics**
 - Expose operational metrics: request rates, error rates, latency, resource usage.
 - Integrate with monitoring tools (e.g., **Prometheus, Google Cloud Monitoring**).
 - **Auditing**
 - Maintain audit trails for critical events:
 - Task creation, state changes, sensitive data access, high-impact operations.
- ✓ Ensures A2A systems are **transparent, debuggable, secure, and enterprise-ready**.

6. API Management & Governance

- **Centralized Policy Enforcement** → Apply security rules (auth, rate limits, quotas) consistently.
- **Traffic Management** → Handle load balancing, routing, and request mediation.
- **Analytics & Reporting** → Gain insights into agent usage, performance, and trends.
- **Developer Portals** → Enable agent discovery, share documentation (Agent Cards), and simplify onboarding.

✅ Ensures **secure, scalable, and well-governed** A2A deployments across enterprises and external ecosystems.

Streaming & Asynchronous Operations in A2A

- **Why needed** → AI tasks can be **long-running, multi-step, incremental, or need human input**.
- **A2A support** → Provides built-in mechanisms to manage these async interactions.
- **Streaming (SSE)** → Clients can stay connected to receive **live updates & partial results**.
- **Push Notifications** → For very long tasks, servers can **notify clients asynchronously** via webhooks.
- **Polling** → Clients can periodically check task status until completion.

✅ Ensures agents can **deliver updates reliably** — whether clients are always online or occasionally disconnected.

1. Streaming with Server-Sent Events (SSE)

Purpose

- For tasks that need **real-time updates** or produce **incremental results** (e.g., long documents, media, partial responses).
- Ideal when the client can keep an **active HTTP connection** open.

How It Works

1. **Initiation** → Client sends `message/stream` RPC and subscribes to updates.
2. **Capability Check** → Server must declare `capabilities.streaming: true` in its Agent Card.
3. **Connection** → If accepted, server returns `200 OK` with `Content-Type: text/event-stream`. Connection stays open.
4. **Events Sent** → Server streams JSON-RPC responses (`SendStreamingMessageResponse`) tied to the original request ID.
 - **Task** → The unit of work being processed.
 - **TaskStatusUpdateEvent** → Status changes (e.g., *working* → *input-required* → *completed*). May also include progress messages.
 - **TaskArtifactUpdateEvent** → Delivers artifacts incrementally (supports chunking & reassembly).
5. **Stream Termination** → Server sends `final: true` (task completed/failed/canceled or awaiting input) → closes connection.
6. **Resubscription** → If the connection breaks, client can use `tasks/resubscribe` RPC to reconnect.

✓ When to Use SSE

- Monitoring **long-running tasks** in real time.
- Receiving **large results in chunks** (start processing early).
- **Interactive exchanges** where partial feedback is helpful.
- Apps needing **low-latency updates** from agents.

2. Push Notifications (for Disconnected Scenarios)

Purpose

- For **very long-running tasks** (minutes → days).
- When clients **can't or don't want** to keep a persistent connection (e.g., mobile apps, serverless functions).

⚙️ How It Works

1. **Capability** → Server declares `capabilities.pushNotifications: true` in its Agent Card.
2. **Configuration** → Client provides a **PushNotificationConfig** (webhook info) either:
 - Inside initial request (`message/send` or `message/stream`), or
 - Later via `tasks/pushNotificationConfig/set` .
 - Config includes:
 - `url` : HTTPS webhook endpoint.
 - `token` (optional): Secret for validation (e.g., `X-A2A-Notification-Token`).
 - `authentication` (optional): Defines how the server authenticates to the webhook.
3. **Trigger** → Server sends notifications on **significant task updates** (e.g., completed, failed, input-required).
4. **Payload** → Minimal (Task ID + new state) or richer (summary/full task object).
5. **Client Action** → On receiving notification:
 - Validate authenticity & relevance.
 - Call `tasks/get` with Task ID → fetch full details/artifacts.

Push Notification Service (Webhook Receiver)

- Runs on client side (or a managed service).
- Responsibilities:
 - Verify the notification source & token.
 - Route it to the correct system (e.g., app logic, queue, API, mobile gateway).
- Can be simple (direct app endpoint) or enterprise-grade (secure, routed, monitored).

✓ **Best For**

- Disconnected clients.
- Tasks where **updates are rare but critical**.
- Enterprise workflows needing **secure async notifications**.

Security Considerations for Push Notifications

Push Notification Security (Server Side)

 **Risk** → Malicious clients could abuse webhook URLs to trigger:

- SSRF (Server-Side Request Forgery)
- DDoS amplification attacks

A2A Server Responsibilities

1. Webhook URL Validation

- Do not blindly send POSTs to any client-provided URL.

2. Mitigation Strategies

- **Allowlisting** → Only accept webhook URLs from trusted domains/IPs.
- **Ownership Verification (Challenge–Response)** →
 - Server sends a `validationToken` in a test request (e.g., GET/OPTIONS).
 - Webhook must echo/confirm token → proves ownership & reachability.
 - Example shown in A2A Python samples.
- **Network Controls** → Restrict outbound traffic using **egress firewalls** or **network policies** to approved destinations only.

🔑 Push Notification Security (Client Webhook Authentication)

📌 Requirement

- The **A2A Server must authenticate** when sending POST requests to the client's webhook.
- Method depends on what the client specifies in `PushNotificationConfig.authentication`.

🔒 Common Schemes

1. Bearer Tokens (OAuth 2.0)

- Server gets an access token (e.g., via OAuth client credentials flow).
- Sends:

```
makefile
```

```
Authorization: Bearer <token>
```

2. API Keys

- Pre-shared key added in a header (e.g., `X-API-Key`).

3. HMAC Signatures

- Server signs request payload with a shared secret.
- Includes signature header (e.g., `X-Hub-Signature`).
- Webhook verifies signature for authenticity.

4. Mutual TLS (mTLS)

- Server presents a client TLS certificate.
- Webhook infrastructure validates certificate.

✅ Ensures that **only authorized servers** can send notifications, preventing spoofing or malicious injections.

Client Webhook Receiver Security (When Receiving Notifications from A2A Server)

Authenticating the A2A Server

Requirement

- The webhook endpoint must **verify every incoming notification** to confirm it came from the **legitimate A2A Server**, not an imposter.

Verification Methods

1. JWTs (Bearer Tokens)

- Validate JWT signature against server's trusted public key (e.g., from a JWKS endpoint).
- Check standard claims:
 - `iss` → trusted issuer
 - `aud` → your webhook identifier
 - `iat`, `exp` → valid issue & expiry times

2. HMAC Signatures

- Recompute signature over payload using shared secret.
- Compare with signature header (e.g., `X-Hub-Signature`).

3. API Keys

- Validate that provided API key matches a known/valid one.

4. Opaque Token Validation

- If `PushNotificationConfig.token` was provided, check that incoming request includes the **exact token** (e.g., `X-A2A-Notification-Token`).
- Ensures notification is tied to the correct **client context & task**.

 Together, these checks protect against **spoofed notifications**, **replay attacks**, and **unauthorized access**.

Preventing Replay Attacks

1. Timestamps

- Include a timestamp (`iat` in JWT or custom header).
- Webhook rejects notifications older than a short window (e.g., a few minutes).
- Timestamp must be part of the **signed payload** for integrity.

2. Nonces / Unique IDs

- Use unique, single-use IDs (`nonce` , event ID, or JWT `jti`).
- Webhook tracks received IDs to block duplicates.

3. Secure Key Management & Rotation

- Apply strong **key management practices**.
- Rotate cryptographic keys regularly.
- For JWT/mTLS → use **JWKS endpoint** to publish public keys.
 - Webhooks fetch correct key dynamically → smooth rotations, stronger security.

 Together, these mechanisms protect against **replay, duplication, and stale notification exploits**.

Example Asymmetric Key Flow (JWT + JWKS)

1. Client Setup

- Client sets `PushNotificationConfig` with `authentication.schemes: ["Bearer"]`.
- May also specify expected `issuer (iss)` or `audience (aud)` for JWTs.

2. A2A Server – Sending Notification

- Generates a **JWT**, signs with its **private key**.
- JWT includes claims:
 - `iss` → issuer
 - `aud` → target webhook
 - `iat` → issued at
 - `exp` → expiration
 - `jti` → unique JWT ID
 - `taskId` → task identifier
- JWT header: `alg` (algorithm) + `kid` (key ID).
- Server exposes **public keys** via a **JWKS endpoint**.

3. Client Webhook – Receiving Notification

1. Extract JWT from `Authorization: Bearer <token>`.
2. Read `kid` from JWT header.
3. Fetch matching **public key** from the server's **JWKS endpoint** (cache keys for efficiency).
4. Verify JWT signature with the public key.
5. Validate claims: `iss`, `aud`, `iat`, `exp`, `jti`.
6. If configured, check `PushNotificationConfig.token` for extra binding to the task/client.

✅ This ensures **strong authentication, replay protection, and secure key rotation** using industry-standard JWT + JWKS practices.

What is Agent Development Kit

Agent Development Kit (ADK) is a flexible and modular framework for **developing and deploying AI agents**. While optimized for Gemini and the Google ecosystem, ADK is **model-agnostic, deployment-agnostic**, and is built for **compatibility with other frameworks**.

ADK was designed to make agent development feel more like software development, to make it easier for developers to create, deploy, and orchestrate agentic architectures that range from simple tasks to complex workflows.

Get started:

Python

Java

```
pip install google-adk
```

Create & activate virtual environment

We recommend creating a virtual Python environment using [venv](#):

```
python -m venv .venv
```

Now, you can activate the virtual environment using the appropriate command for your operating system and environment:

```
# Mac / Linux
source .venv/bin/activate

# Windows CMD:
.venv\Scripts\activate.bat

# Windows PowerShell:
.venv\Scripts\Activate.ps1
```

Install ADK ¶

```
pip install google-adk
```

(Optional) Verify your installation:

```
pip show google-adk
```

Agent Development Kit (ADK)

Build, Evaluate and Deploy agents, seamlessly!

ADK is designed to empower developers to build, manage, evaluate and deploy AI-powered agents. It provides a robust and flexible environment for creating both conversational and non-conversational agents, capable of handling complex tasks and workflows.

Discover, build and deploy agents

Agent Development Kit

Client side SDK to define multi-agent applications for complex, real world scenarios

Agents

Tool

Orchestration

Callbacks

Bidirectional
Streaming

Session
Management

Evaluation

Deployment

Artifact
Management

Memory

Code
Execution

Planning

Debugging

Trace

Models

Core Concepts

ADK is built around a few key primitives and concepts that make it powerful and flexible. Here are the essentials:

- **Agent:** The fundamental worker unit designed for specific tasks. Agents can use language models (`LlmAgent`) for complex reasoning, or act as deterministic controllers of the execution, which are called "[workflow agents](#)" (`SequentialAgent` , `ParallelAgent` , `LoopAgent`).
- **Tool:** Gives agents abilities beyond conversation, letting them interact with external APIs, search information, run code, or call other services.
- **Callbacks:** Custom code snippets you provide to run at specific points in the agent's process, allowing for checks, logging, or behavior modifications.

- **Session Management (Session & State):** Handles the context of a single conversation (Session), including its history (Events) and the agent's working memory for that conversation (State).
- **Memory:** Enables agents to recall information about a user across *multiple* sessions, providing long-term context (distinct from short-term session State).
- **Artifact Management (Artifact):** Allows agents to save, load, and manage files or binary data (like images, PDFs) associated with a session or user.
- **Code Execution:** The ability for agents (usually via Tools) to generate and execute code to perform complex calculations or actions.
- **Planning:** An advanced capability where agents can break down complex goals into smaller steps and plan how to achieve them like a ReAct planner.

- **Models:** The underlying LLM that powers LlmAgent s, enabling their reasoning and language understanding abilities.
- **Event:** The basic unit of communication representing things that happen during a session (user message, agent reply, tool use), forming the conversation history.
- **Runner:** The engine that manages the execution flow, orchestrates agent interactions based on Events, and coordinates with backend services.

Key Capabilities

ADK offers several key advantages for developers building agentic applications:

1. **Multi-Agent System Design:** Easily build applications composed of multiple, specialized agents arranged hierarchically. Agents can coordinate complex tasks, delegate sub-tasks using LLM-driven transfer or explicit `AgentTool` invocation, enabling modular and scalable solutions.
2. **Rich Tool Ecosystem:** Equip agents with diverse capabilities. ADK supports integrating custom functions (`FunctionTool`), using other agents as tools (`AgentTool`), leveraging built-in functionalities like code execution, and interacting with external data sources and APIs (e.g., Search, Databases). Support for long-running tools allows handling asynchronous operations effectively.

3. **Flexible Orchestration:** Define complex agent workflows using built-in workflow agents (`SequentialAgent` , `ParallelAgent` , `LoopAgent`) alongside LLM-driven dynamic routing. This allows for both predictable pipelines and adaptive agent behavior.
4. **Integrated Developer Tooling:** Develop and iterate locally with ease. ADK includes tools like a command-line interface (CLI) and a Developer UI for running agents, inspecting execution steps (events, state changes), debugging interactions, and visualizing agent definitions.
5. **Native Streaming Support:** Build real-time, interactive experiences with native support for bidirectional streaming (text and audio). This integrates seamlessly with underlying capabilities like the [Multimodal Live API for the Gemini Developer API](#) (or for [Vertex AI](#)), often enabled with simple configuration changes.

6. **Built-in Agent Evaluation:** Assess agent performance systematically. The framework includes tools to create multi-turn evaluation datasets and run evaluations locally (via CLI or the dev UI) to measure quality and guide improvements.
7. **Broad LLM Support:** While optimized for Google's Gemini models, the framework is designed for flexibility, allowing integration with various LLMs (potentially including open-source or fine-tuned models) through its `BaseLlm` interface.
8. **Artifact Management:** Enable agents to handle files and binary data. The framework provides mechanisms (`ArtifactService` , context methods) for agents to save, load, and manage versioned artifacts like images, documents, or generated reports during their execution.

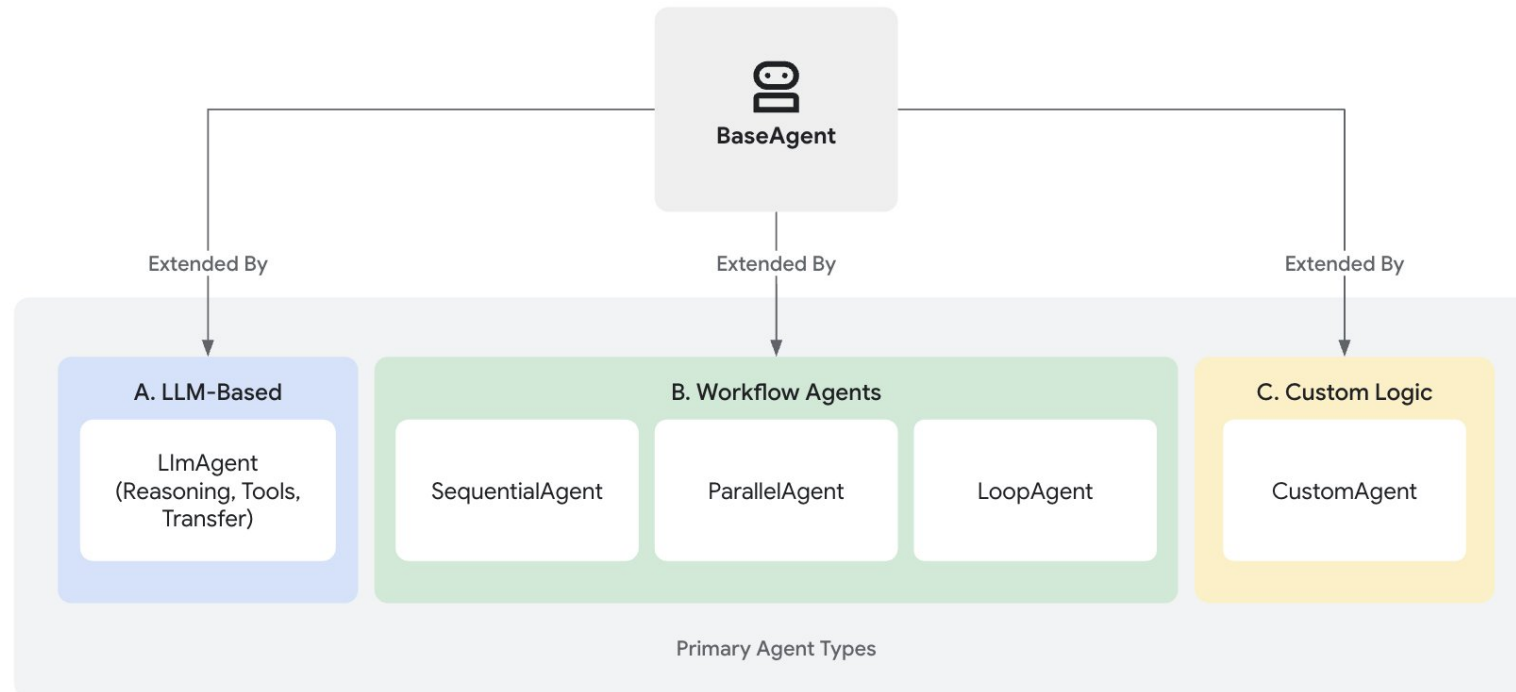
9. **Extensibility and Interoperability:** ADK promotes an open ecosystem. While providing core tools, it allows developers to easily integrate and reuse tools from other popular agent frameworks including LangChain and CrewAI.
10. **State and Memory Management:** Automatically handles short-term conversational memory (`State` within a `Session`) managed by the `SessionService` . Provides integration points for longer-term `Memory` services, allowing agents to recall user information across multiple sessions.



Agents

In the Agent Development Kit (ADK), an **Agent** is a self-contained execution unit designed to act autonomously to achieve specific goals. Agents can perform tasks, interact with users, utilize external tools, and coordinate with other agents.

The foundation for all agents in ADK is the `BaseAgent` class. It serves as the fundamental blueprint. To create functional agents, you typically extend `BaseAgent` in one of three main ways, catering to different needs – from intelligent reasoning to structured process control.



Core Agent Categories

The **Agent Development Kit (ADK)** provides three main categories of agents for building advanced applications:

1. **LLM Agents** (*LlmAgent, Agent*)

Powered by Large Language Models (LLMs), these agents can interpret natural language, reason, plan, and dynamically decide which tools or actions to take. They are best suited for flexible, language-driven tasks that require adaptability and intelligence.

2. **Workflow Agents** (*SequentialAgent, ParallelAgent, LoopAgent*)

These agents orchestrate other agents in structured patterns—such as sequences, parallel executions, or loops—without relying on LLMs for flow control. Ideal for deterministic, predictable workflows.

3. **Custom Agents**

Built by extending `BaseAgent`, these agents let developers define specialized logic, custom control flows, or unique integrations beyond the standard categories. Perfect for highly tailored use cases.

Choosing the Right Agent Type

The following table provides a high-level comparison to help distinguish between the agent types. As you explore each type in more detail in the subsequent sections, these distinctions will become clearer.

Feature	LLM Agent (LlmAgent)	Workflow Agent	Custom Agent (BaseAgent subclass)
Primary Function	Reasoning, Generation, Tool Use	Controlling Agent Execution Flow	Implementing Unique Logic/Integrations
Core Engine	Large Language Model (LLM)	Predefined Logic (Sequence, Parallel, Loop)	Custom Code
Determinism	Non-deterministic (Flexible)	Deterministic (Predictable)	Can be either, based on implementation
Primary Use	Language tasks, Dynamic decisions	Structured processes, Orchestration	Tailored requirements, Specific workflows

Agents Working Together: Multi-Agent Systems

While each agent type serves a distinct purpose, the true power often comes from combining them.

Complex applications frequently employ where:

LLM Agents handle intelligent, language-based task execution.

Workflow Agents manage the overall process flow using standard patterns.

Custom Agents provide specialized capabilities or rules needed for unique integrations.

LLM Agent

LlmAgent (Agent) – Key Highlights

- **Core Role:** Serves as the “*thinking*” engine of ADK applications.
- **Powered by LLMs:** Uses Large Language Models to:
 - Understand natural language
 - Reason and make decisions
 - Generate responses
 - Interact with tools
- **Dynamic Behavior:**
 - **Non-deterministic** (unlike Workflow Agents that follow fixed paths).
 - Decides *on the fly* how to act, which tool to use, or when to hand off to another agent.
- **How to Build Effectively:**
 - Define the agent’s **identity** (role, purpose, personality).
 - Provide **clear instructions** to shape behavior.
 - Equip with the right **tools and capabilities** for the task.

Defining an Agent's Identity and Purpose



1. Name (Required)

- Unique string identifier for the agent.
- Essential in multi-agent systems for delegation and reference.
- Should be **descriptive of its function** (e.g., `customer_support_router`, `billing_inquiry_agent`).
- Avoid reserved names like `user`.

2. Description (Optional, but Recommended in Multi-Agent setups)

- Concise summary of the agent's **capabilities**.
- Helps other LLM agents decide whether to route tasks to it.
- Must be **specific** (e.g., "Handles inquiries about current billing statements" rather than just "Billing agent").

3. Model (Required)

- Defines the **LLM powering the agent's reasoning**.
- Example: `"gemini-2.0-flash"`.
- Impacts **capabilities, cost, and performance**.
- Should be chosen carefully based on application needs.

```
# Example: Defining the basic identity
capital_agent = LlmAgent(
    model="gemini-2.0-flash",
    name="capital_agent",
    description="Answers user questions about the capital city of a given country.
    # instruction and tools will be added next
)
```



Guiding the Agent: Instructions (`instruction`)

The `instruction` parameter is the **most critical element** in shaping an **LlmAgent's** behavior.

What Instructions Define

- **Core Task/Goal** → What the agent should accomplish.
- **Personality/Persona** → Style of interaction (e.g., *"You are a helpful assistant"*, *"You are a witty pirate"*).
- **Constraints** → What the agent *can* or *cannot* do (e.g., *"Only answer about X"*, *"Never reveal Y"*).
- **Tool Usage** → How and when to use tools, with explanations of purpose and circumstances.
- **Output Format** → Required response style (e.g., JSON, bulleted list).

Tips for Writing Effective Instructions

- **Be Clear & Specific** → Avoid vague wording. State exact actions/outcomes.
- **Use Markdown** → Improves readability for structured/complex guidance.
- **Provide Examples (Few-Shot)** → Show desired formats or responses.
- **Guide Tool Use** → Specify *when* and *why* tools should be applied, not just their names.

Dynamic Instruction Templates

- Use `{var}` → Inserts the value of a state variable named `var`.
- Use `{artifact.var}` → Inserts the text content of an artifact named `var`.
- Missing values raise errors unless marked optional with `?`.
 - Example: `{var?}` → Ignores error if `var` does not exist.

⚡ **Key Insight:** The `instruction` acts as the **blueprint for agent behavior**—it controls *what it does, how it speaks, which tools it uses, and what the output looks like*.



```
# Example: Adding instructions
capital_agent = LlmAgent(
    model="gemini-2.0-flash",
    name="capital_agent",
    description="Answers user questions about the capital city of a given country.",
    instruction="""You are an agent that provides the capital city of a country.
When a user asks for the capital of a country:
1. Identify the country name from the user's query.
2. Use the `get_capital_city` tool to find the capital.
3. Respond clearly to the user, stating the capital city.
Example Query: "What's the capital of {country}?"
Example Response: "The capital of France is Paris."
""",
    # tools will be added next
)
```

Equipping the Agent: Tools (`tools`)

- **Purpose of Tools**
 - Extend the agent's abilities beyond the LLM's built-in knowledge.
 - Let the agent interact with the outside world.
 - Enable tasks like calculations, fetching real-time data, or executing actions.
- **Definition**
 - `tools` is **optional** – a list of capabilities the agent can use.
- **Types of Tools**
 1. **Native Functions/Methods**
 - In Python: automatically wrapped as `FunctionTool`.
 - In Java: must be explicitly wrapped with `FunctionTool.create(...)`.
 2. **Custom Tools**
 - Instances of classes inheriting from `BaseTool`.
 3. **Other Agents**
 - Via `AgentTool`, enabling agent-to-agent delegation (used in multi-agent setups).
- **How Tools Are Used**
 - The LLM selects tools by looking at:
 - **Name**
 - **Description** (docstring or `description` field)
 - **Parameter schema**
 - Decides which tool to call based on **conversation context + instructions**.



⚡ **Key Insight:** Tools act as the **agent's hands and eyes**, letting it perform actions, delegate, and fetch data beyond its internal reasoning.

```
# Define a tool function
def get_capital_city(country: str) -> str:
    """Retrieves the capital city for a given country."""
    # Replace with actual logic (e.g., API call, database lookup)
    capitals = {"france": "Paris", "japan": "Tokyo", "canada": "Ottawa"}
    return capitals.get(country.lower(), f"Sorry, I don't know the capital of {country}")

# Add the tool to the agent
capital_agent = LlmAgent(
    model="gemini-2.0-flash",
    name="capital_agent",
    description="Answers user questions about the capital city of a given country.",
    instruction="""You are an agent that provides the capital city of a country...""",
    tools=[get_capital_city] # Provide the function directly
)
```

Advanced Configuration & Control

Fine-Tuning LLM Generation (`generate_content_config`)

- **Purpose:** Adjust how the LLM generates responses.
- **Parameter:**
 - `generate_content_config` (*optional*) → An instance of `google.genai.types.GenerateContentConfig`.
- **What You Can Control:**
 - **temperature** → Controls randomness (higher = more creative, lower = more deterministic).
 - **max_output_tokens** → Sets the maximum response length.
 - **top_p** → Nucleus sampling (limits word choices to top probability mass).
 - **top_k** → Limits word choices to the top *k* tokens.
 - **safety settings** → Apply filters or restrictions to outputs.

⚡ **Key Insight:** `generate_content_config` lets you **fine-tune style, length, and safety** of responses, making the agent's outputs more predictable or more creative depending on your needs.

```
from google.genai import types

agent = LlmAgent(
    # ... other params
    generate_content_config=types.GenerateContentConfig(
        temperature=0.2, # More deterministic output
        max_output_tokens=250
    )
)
```


Structuring Data (`input_schema`, `output_schema`, `output_key`)

- **Why:** Useful when you need **structured data exchange** with an LLM Agent instead of free text.

Key Components

1. `input_schema` (Optional)

- Defines the **expected input format**.
- User (or another agent) must provide a JSON string that matches this schema.
- Instructions should guide how to supply this structured input.

2. `output_schema` (Optional)

- Defines the **desired output format**.
- Agent's final response must be a JSON string matching this schema.

3. `output_key` (Optional)

- A string key where the agent's final response is saved in the **session's state dictionary**.
- Helps in passing results between **agents** or **workflow steps**.
- Example:
 - **Python** → `session.state[output_key] = agent_response_text`
 - **Java** → `session.state().put(outputKey, agentResponseText)`

⚡ **Key Insight:** These parameters make the agent's inputs/outputs **consistent, machine-readable, and easy to reuse** in multi-agent workflows.

```
from pydantic import BaseModel, Field
```

```
class CapitalOutput(BaseModel):  
    capital: str = Field(description="The capital of the country.")
```

```
structured_capital_agent = LlmAgent(  
    # ... name, model, description  
    instruction="""You are a Capital Information Agent. Given a country, respond ONLY with a JSON  
object containing the capital. Format: {"capital": "capital_name"}""",  
    output_schema=CapitalOutput,                # Enforce JSON output  
    output_key="found_capital"                   # Store result in state['found_capital']  
    # Cannot use tools=[get_capital_city] effectively here  
)
```



Managing Context (`include_contents`)

- **Purpose:** Controls whether the agent has access to **prior conversation history**.
- **Parameter:** `include_contents` (*optional, default = `'default'`*)
- **Options:**
 1. `'default'` → Agent receives the **relevant conversation history** (good for continuity).
 2. `'none'` → Agent ignores all prior history, relying only on the **current instruction and input** (useful for **stateless tasks** or when enforcing a specific context).

⚡ **Key Insight:** Use `'default'` for natural conversations, and `'none'` when you need the agent to **focus** only on the present turn.

```
stateless_agent = LlmAgent(  
    # ... other params  
    include_contents='none'  
)
```

Planner

Planner (planner)

- **Purpose:** Enables **multi-step reasoning and planning** before execution.
- **Parameter:** `planner` (*optional*) → Assign a `BasePlanner` instance.

Types of Planners

1. BuiltInPlanner

- Uses the model's **built-in planning features** (e.g., *Gemini's thinking*).
- Supports advanced reasoning before producing a response.

Key Settings

- `thinking_budget` → Defines how many *thinking tokens* the model can use for internal reasoning.
- `include_thoughts` → Controls whether the model's **raw thoughts/internal reasoning** are exposed in the response.

⚡ **Key Insight:** A planner makes the agent **smarter at breaking down complex tasks**, with fine control over how much reasoning is done and whether that reasoning is visible.



```
from google.adk import Agent
from google.adk.planners import BuiltInPlanner
from google.genai import types

my_agent = Agent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,
            thinking_budget=1024,
        )
    ),
    # ... your tools here
)
```


Planner (planner)

- **Purpose:** Enables **multi-step reasoning and planning** before execution.
 - **Parameter:** `planner` (optional) → Assign a `BasePlanner` instance.
-

Types of Planners

1. BuiltInPlanner

- Uses the model's **native planning abilities** (e.g., *Gemini Thinking*).
- `thinking_budget` → Sets the number of tokens for internal reasoning.
- `include_thoughts` → Decides whether raw reasoning is exposed in the response.

2. PlanReActPlanner

- Enforces a **structured output style**:
 - First → **Plan**
 - Then → **Execute actions** (e.g., tool calls)
 - Finally → **Provide reasoning**
- Ideal for models **without built-in “thinking” features**.

```
from google.adk import Agent
from google.adk.planners import PlanReActPlanner

my_agent = Agent(
    model="gemini-2.0-flash",
    planner=PlanReActPlanner(),
    # ... your tools here
)
```

⚡ Key Insight:

- Use **BuiltInPlanner** if the model already supports advanced internal reasoning.
- Use **PlanReActPlanner** if you need a **structured plan-act-reason workflow** for clearer, stepwise execution.

Planner Comparison

Feature	BuiltInPlanner	PlanReActPlanner
Purpose	Uses the model's native planning abilities (e.g., Gemini Thinking).	Enforces a structured plan → act → reason workflow.
Best For	Models with built-in "thinking" features.	Models that lack native reasoning capabilities.
Reasoning Style	Internal reasoning guided by thinking tokens .	Explicit stepwise reasoning shown in output.
Key Parameters	- <code>thinking_budget</code> : limit of tokens for internal thought. - <code>include_thoughts</code> : whether to expose raw thoughts.	No special params—follows the fixed Plan → Action → Reason structure.
Transparency	Can hide or show reasoning (<code>include_thoughts</code>).	Always shows reasoning in structured form.
Use Case Example	Complex multi-step tasks where reasoning can stay hidden (e.g., summarization + retrieval).	Teaching or debugging workflows where visible reasoning is helpful (e.g., tool-based QA systems).

⚡ Quick Takeaway:

- **BuiltInPlanner** → Smarter, more natural if model supports reasoning.
- **PlanReActPlanner** → More **explicit and structured**, best when reasoning must be **visible and controlled**.

The agent's response will follow a structured format:

```
[user]: ai news
[google_search_agent]: /*PLANNING*/
1. Perform a Google search for "latest AI news" to get current updates and headlines.
2. Synthesize the information from the search results to provide a summary of recent AI news.

/*ACTION*/
/*REASONING*/
The search results provide a comprehensive overview of recent AI news, covering various topics such as machine learning, natural language processing, and robotics.

/*FINAL_ANSWER*/
Here's a summary of recent AI news:
....
```

Code Execution

Code Executor (`code_executor`)

- **Purpose:** Lets the agent **run code blocks** that appear in the LLM's response.
 - **Parameter:** `code_executor` (*optional*) → Provide a `BaseCodeExecutor` instance.
 - **How It Works:**
 - If the LLM outputs code (e.g., Python snippet), the executor can run it.
 - Useful for **dynamic calculations, data processing, or trying out generated code.**
 - **Benefit:** Extends the agent's ability beyond just reasoning → it can **actually execute and verify code.**
-

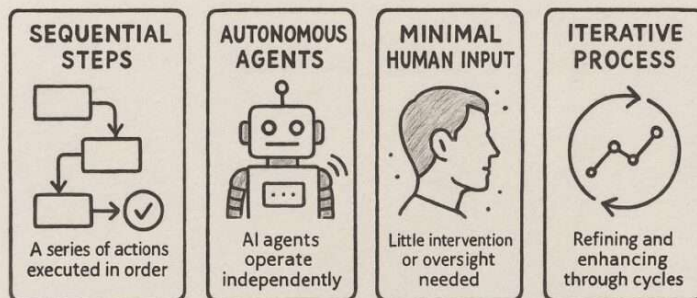
⚡ **Key Insight:** With a code executor, the agent becomes **interactive and action-oriented**, not just a text generator.

[Code executor](#)

[File reference - LLMagent](#) All

Workflow Agents

AGENTIC WORKFLOWS



Workflow Agents

- **Purpose:**
 - Specialized agents in ADK that **orchestrate the execution flow** of sub-agents.
 - Define **how and when** other agents should run.
- **Role:**
 - Focus only on **control flow management**, not reasoning.
 - Ensure tasks follow a **structured process**.
- **Difference from LLM Agents:**
 - **LLM Agents** → Use language models for dynamic reasoning and flexible decision-making.
 - **Workflow Agents** → Follow **predefined logic** (sequence, parallel, loop).
- **Key Benefit:**
 - Execution is **deterministic and predictable**, making them reliable for structured workflows.

⚡ **Key Insight:** Workflow Agents act as the **directors of execution**, while LLM Agents act as the **thinkers**.

Why Use Workflow Agents?

- **Explicit Control** → Define exactly *how tasks or agents should run*.
- **Predictability** → Execution flow is **guaranteed** by the agent type (sequential, parallel, loop).
- **Reliability** → Tasks always run in the **required order or pattern**.
- **Structure** → Build complex processes by combining multiple agents within clear control flows.
- **Flexibility** → Sub-agents can include **LLM Agents**, blending:
 - **Deterministic orchestration** (workflow agent)
 - **Dynamic reasoning** (LLM agent)

⚡ **Key Insight:** Workflow Agents let you **combine strict process control with LLM intelligence**, making them ideal for structured yet intelligent applications.

Sequential agents

Sequential Agent

- **What It Is:**
 - A **workflow agent** that executes its sub-agents in a **strict, fixed order**.
- **When to Use:**
 - When tasks must always run in a **specific sequence**.
 - Example:
 1. **Get Page Contents** →
 2. **Summarize Page**(You can't summarize before fetching the content.)
- **Key Traits:**
 - **Deterministic** → No randomness; always follows the defined order.
 - **Not powered by an LLM** → It just controls execution flow.
 - Sub-agents or tools inside it **may or may not use LLMs**.

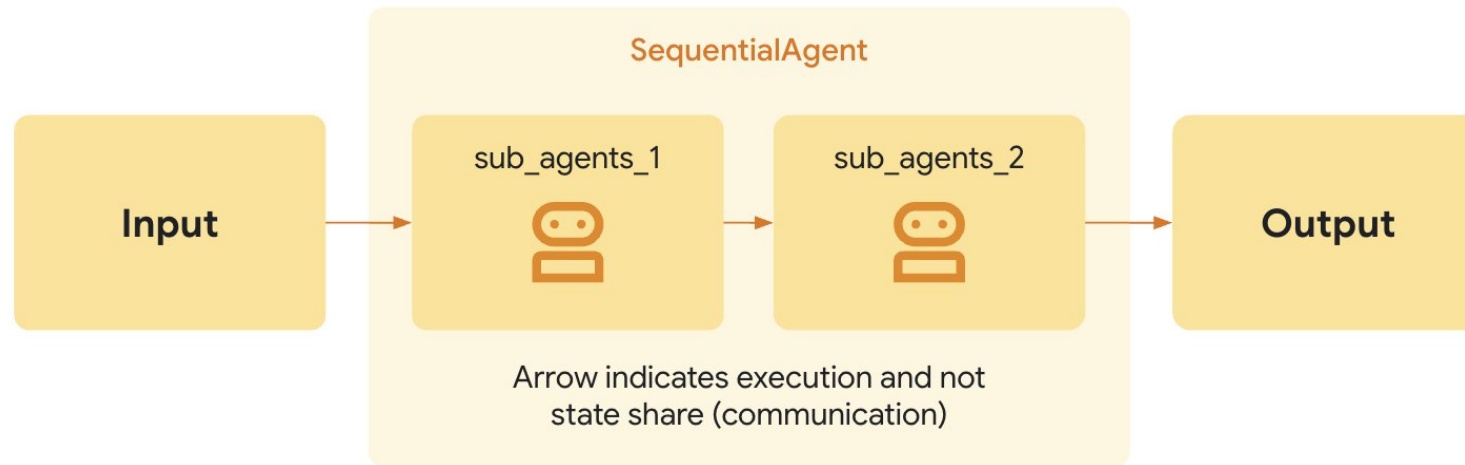
⚡ **Key Insight:** Use a **SequentialAgent** whenever the **order of execution is critical**.

How SequentialAgent Works

When you call the `RunAsync` method, the SequentialAgent does the following:

1. **Iteration** → Goes through the list of sub-agents in the exact order they were defined.
2. **Execution** → For each sub-agent, it calls that sub-agent's `RunAsync` method.

⚡ **Key Insight:** The SequentialAgent simply **walks through sub-agents step by step**, ensuring each one runs in order before moving to the next.



Example: Code Development Pipeline

A `SequentialAgent` can orchestrate a simple code development process:

1. Code Writer Agent

- Type: **LLM Agent**
- Task: Generates the **initial code** from a given specification.

2. Code Reviewer Agent

- Type: **LLM Agent**
- Task: Reviews the generated code for **errors, style issues, and best practices**.
- Input: Takes the **output of the Code Writer Agent**.

3. Code Refactorer Agent

- Type: **LLM Agent**
- Task: Refactors the reviewed code to **fix issues and improve quality**.
- Input: Uses both the **code** and the **review comments**.

[subagent.py](#)

⚡ Key Insight:

Using a **`SequentialAgent`**, you can build a **pipeline** where each agent's output flows into the next — ensuring code is written, reviewed, and improved in strict order.

```
SequentialAgent(sub_agents=[CodeWriterAgent, CodeReviewerAgent, CodeRefactorerAgent])
```

This ensures the code is written, *then* reviewed, and *finally* refactored, in a strict, dependable order. **The output from each sub-agent is passed to the next by storing them in state via .**

Loop agents



Loop Agent

- **What It Is**
 - A **workflow agent** that runs its sub-agents in a **loop**.
 - Executes repeatedly for a **set number of iterations** or until a **stop condition** is met.
- **When to Use**
 - Workflows that require **repetition** or **iterative refinement**.
 - Example:
 - **Generate Image** → create food image.
 - **Count Food Items** → check if number of items matches the request.
 - Repeat until the count is correct (e.g., exactly 5 bananas) or until iteration limit is reached.
- **Key Traits**
 - **Deterministic** → Always follows the loop logic exactly.
 - **Not powered by an LLM** → Focused only on **execution control**.
 - Sub-agents/tools inside may or may not use LLMs.

⚡ **Key Insight:** Use a **LoopAgent** when your workflow needs **retrying or refining results** until a condition is satisfied.

How LoopAgent Works

When you call the `RunAsync` method, the LoopAgent does the following:

1. Sub-Agent Execution

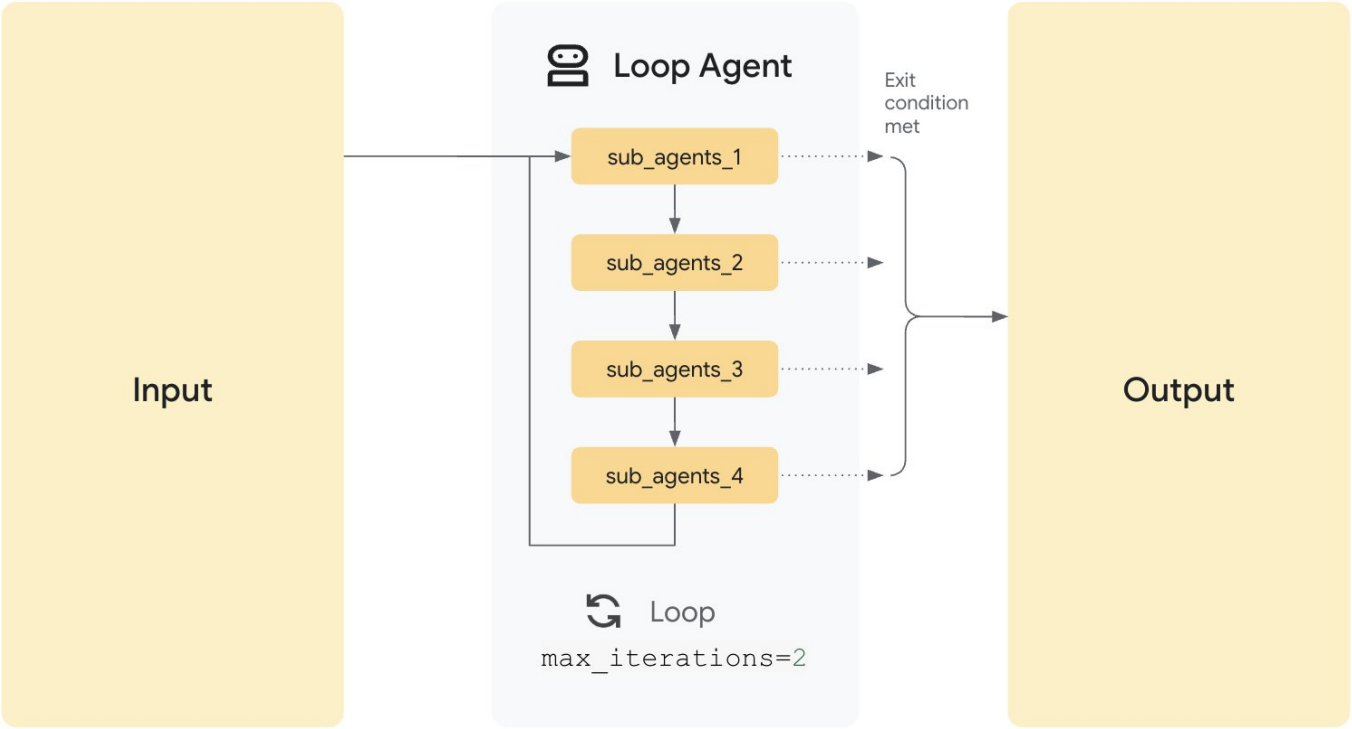
- Iterates through the sub-agent list in order.
- For each one, calls its `RunAsync` method.

2. Termination Check

- The LoopAgent itself does **not** decide when to stop.
- You must define a **termination mechanism** to avoid infinite loops.

Common strategies:

- **Max Iterations** → Set a maximum loop count; stops after reaching it.
- **Condition Check (via sub-agent)** → A sub-agent evaluates progress (e.g., *"Is the output correct?"*).
 - If condition is met, it signals termination (e.g., event, flag, or return value).



⚡ **Key Insight:** The LoopAgent provides **controlled repetition**, but you must design the **stop rule** to make it safe and effective.

Example: Iterative Document Improvement

A **LoopAgent** can manage a process where a document is refined step by step:

1. Writer Agent (LLM Agent)

- Generates or improves a draft on the given topic.

2. Critic Agent (LLM Agent)

- Reviews the draft and identifies areas for improvement.

3. Loop Execution

- `LoopAgent(sub_agents=[WriterAgent, CriticAgent], max_iterations=5)`
- Runs Writer → Critic repeatedly.

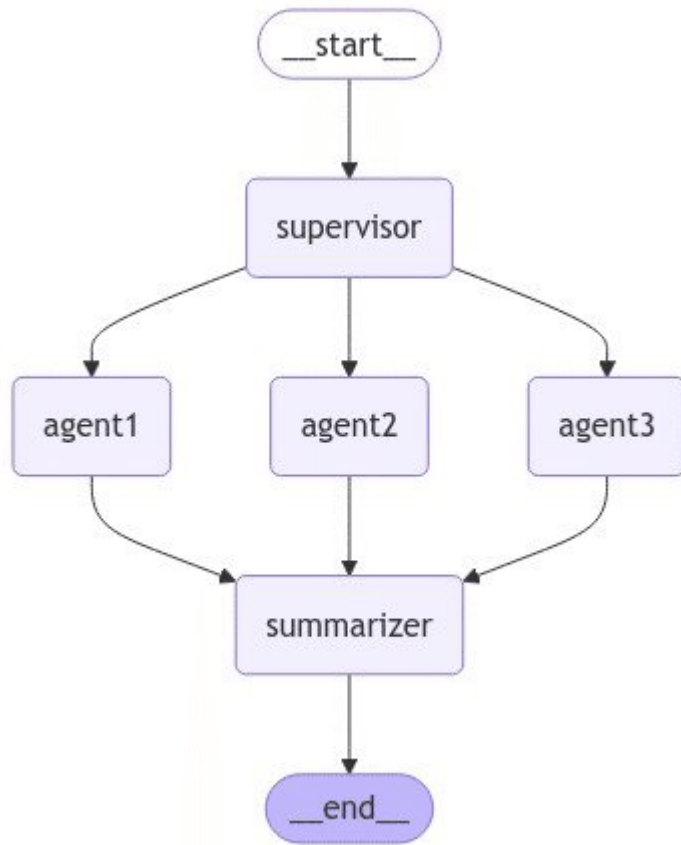
4. Termination Options

- **Critic signals STOP** → Ends early when document quality is good enough.
- **Max Iterations (5)** → Stops after 5 cycles if no stop signal is given.
- **External Logic** → Custom stop conditions (e.g., based on user feedback).

[Ref -
LoopAgent](#)

⚡ **Key Insight:** A LoopAgent enables **iterative refinement** with built-in safety — either by **quality-based stop signals** or by a **fixed iteration cap**.

Parallel agents



Parallel Agent

- **What It Is**
 - A **workflow agent** that runs its sub-agents **concurrently**.
 - Designed to speed up workflows where tasks can be done at the same time.
- **When to Use**
 - **Independent tasks** with no dependencies.
 - **Resource-intensive tasks** where parallel execution saves time.
 - Example: Multiple data fetches, API calls, or analysis steps that don't rely on each other.
- **Key Traits**
 - **Deterministic** → Always executes in parallel, not random.
 - **Not powered by an LLM** → Only manages execution.
 - Sub-agents/tools inside it may or may not use LLMs.

⚡ **Key Insight:** Use a **ParallelAgent** when you want to **maximize speed** by running independent tasks **side by side** instead of in sequence.

ParallelAgent – Example & How It Works

Example Use Case

- Ideal for **multi-source data retrieval** or **heavy computations**.
 - Works best when tasks are **independent** and don't need to share state or exchange information.
-

How It Works (run_async)

1. Concurrent Execution

- Calls `run_async()` for each sub-agent in the list **at the same time**.

2. Independent Branches

- Each sub-agent runs in its **own branch**.
- No automatic sharing of conversation history or state between agents.

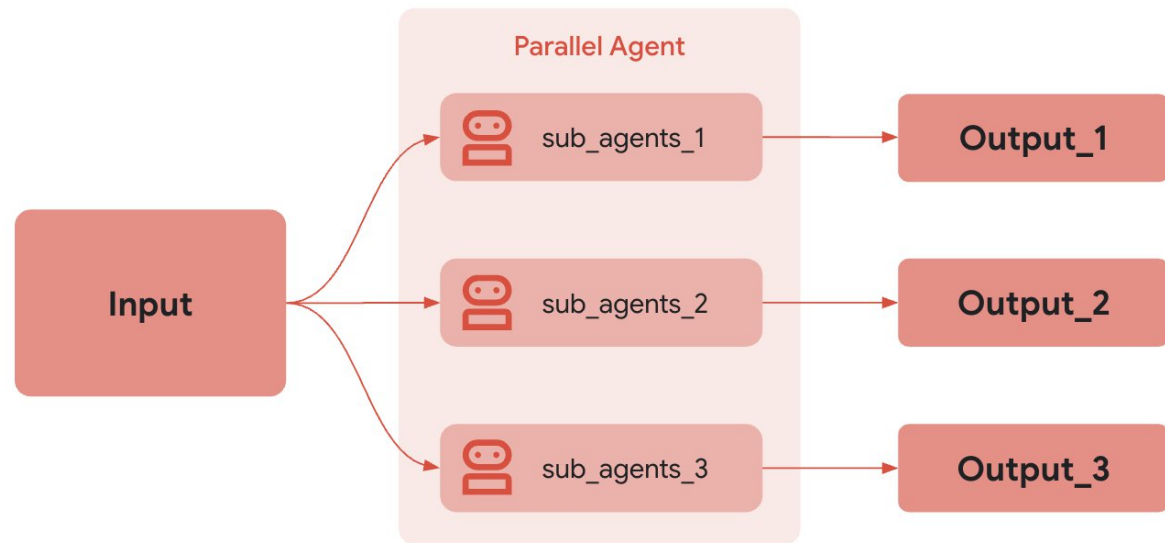
3. Result Collection

- ParallelAgent collects the results after all sub-agents finish.
 - Results are usually returned as a **list or set of events**.
 - The **order of results is not guaranteed** (non-deterministic).
-

⚡ **Key Insight:** ParallelAgent is perfect when you need **speed through concurrency**, but only if your tasks can run **independently** without shared state.

Independent Execution & State Management (ParallelAgent)

- **Default Behavior:**
 - Sub-agents run **independently**.
 - No built-in communication or data sharing between them.
- **If Data Sharing Is Needed** → You must design it explicitly:
 1. **Shared InvocationContext**
 - Pass a common context object to all sub-agents.
 - Acts as a **shared data store**.
 - ⚠️ Must manage **concurrent access** (e.g., locks) to avoid race conditions.
 2. **External State Management**
 - Use an external system (e.g., **database, message queue**) to handle shared state and inter-agent communication.
 3. **Post-Processing**
 - Collect results after all sub-agents finish.
 - Apply coordination or merging logic at the end.



⚡ **Key Insight:** ParallelAgent ensures **speed through concurrency**, but **state sharing is not automatic** — you need to build your own mechanism if agents must coordinate.

Example: Parallel Web Research

- **Goal:** Research multiple topics **at the same time** to finish faster.
- **Agents:**
 - **Researcher Agent 1** → “renewable energy sources”
 - **Researcher Agent 2** → “electric vehicle technology”
 - **Researcher Agent 3** → “carbon capture methods”
- **Execution:**
 - Use `ParallelAgent(sub_agents=[Agent1, Agent2, Agent3])`
 - All three researchers **run concurrently** (start together, work independently).
 - Each agent produces its **own result**.
- **Result Handling:**
 - Collect outputs **separately** after completion.
 - Order of results may **not** be guaranteed.
 - Combine or summarize results in a **post-processing** step if needed.
- **Why Parallel?**
 - **Speed:** Cuts total time vs running one-by-one.
 - **Independence:** No dependencies between topics, so safe to parallelize.
- **Notes:**
 - `ParallelAgent` controls **execution only** (deterministic orchestration).
 - Sub-agents **may** use LLMs; `ParallelAgent` itself does **not**.
 - If you need shared state/communication, add it explicitly (shared context, DB/queue, or post-processing).

ref - [parallel.py](#)

Custom agents

Custom agents provide the ultimate flexibility in ADK, allowing you to define **arbitrary orchestration logic** by inheriting directly from `BaseAgent` and implementing your own control flow.

This goes beyond the predefined patterns of `SequentialAgent`, `LoopAgent`, and `ParallelAgent`, enabling you to build highly specific and complex agentic workflows

What is a Custom Agent?

A Custom Agent is essentially any class you create that inherits from `google.adk.agents.BaseAgent` and implements its core execution logic within the `_run_async_impl` asynchronous method.

You have complete control over how this method calls other agents (sub-agents), manages state, and handles events.

Why Use Custom Agents?

Standard workflow agents (Sequential, Loop, Parallel) cover most needs — but use a **Custom Agent** when you need more advanced or flexible orchestration:

1. Conditional Logic

- Choose different sub-agents or paths based on **runtime conditions** or prior results.

2. Complex State Management

- Maintain or update **state** in sophisticated ways beyond simple step-to-step passing.

3. External Integrations

- Directly call **APIs, databases, or custom libraries** within the workflow control.

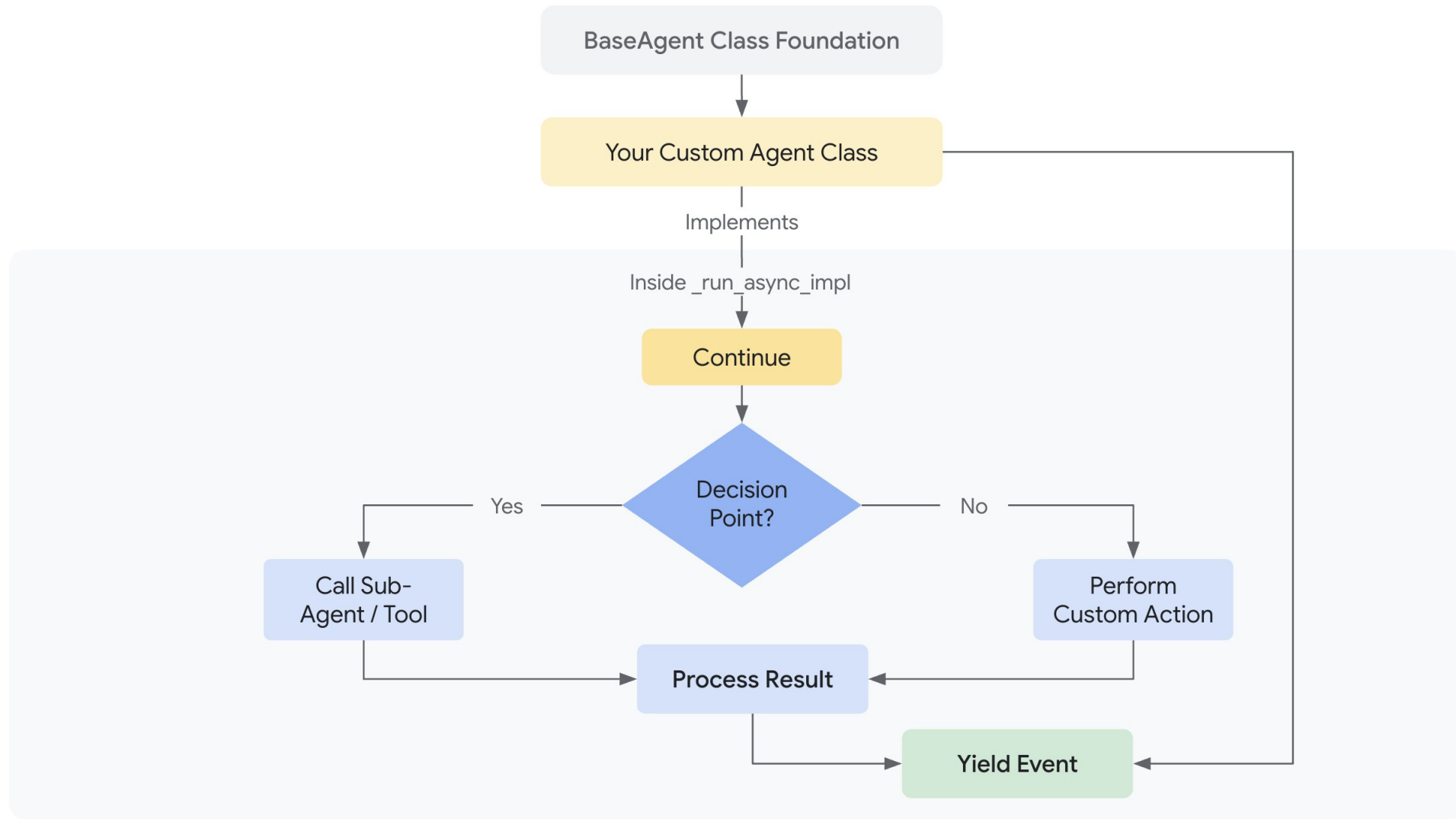
4. Dynamic Agent Selection

- Decide at runtime which sub-agents to run based on **input or context**.

5. Unique Workflow Patterns

- Create orchestration flows that don't fit **sequential, parallel, or loop structures**.

⚡ **Key Insight:** Custom Agents give you **full flexibility** to design orchestration that matches **complex, dynamic, or highly specialized workflows**.



Implementing Custom Logic:

The core of any custom agent is the method where you define its unique asynchronous behavior.

This method allows you to orchestrate sub-agents and manage the flow of execution.

Python

Java

The heart of any custom agent is the `_run_async_impl` method. This is where you define its unique behavior.

- **Signature:** `async def _run_async_impl(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]:`
- **Asynchronous Generator:** It must be an `async def` function and return an `AsyncGenerator`. This allows it to `yield` events produced by sub-agents or its own logic back to the runner.
- **ctx (InvocationContext):** Provides access to crucial runtime information, most importantly `ctx.session.state`, which is the primary way to share data between steps orchestrated by your custom agent.

Key Capabilities within the Core Asynchronous Method:

1. **Calling Sub-Agents:** You invoke sub-agents (which are typically stored as instance attributes like `self.my_llm_agent`) using their `run_async` method and yield their events:

```
async for event in self.some_sub_agent.run_async(ctx):  
    # Optionally inspect or log the event  
    yield event # Pass the event up
```

2. **Managing State:** Read from and write to the session state dictionary (`ctx.session.state`) to pass data between sub-agent calls or make decisions:

```
# Read data set by a previous agent  
previous_result = ctx.session.state.get("some_key")  
  
# Make a decision based on state  
if previous_result == "some_value":  
    # ... call a specific sub-agent ...  
else:  
    # ... call another sub-agent ...  
  
# Store a result for a later step (often done via a sub-agent's output_key)  
# ctx.session.state["my_custom_result"] = "calculated_value"
```

3. **Implementing Control Flow:** Use standard Python constructs (`if / elif / else`, `for / while` loops, `try / except`) to create sophisticated, conditional, or iterative workflows involving your sub-agents.

Managing Sub-Agents and State

Typically, a custom agent orchestrates other agents (like `LlmAgent` , `LoopAgent` , etc.).

- **Initialization:** You usually pass instances of these sub-agents into your custom agent's constructor and store them as instance fields/attributes (e.g., `this.story_generator = story_generator_instance` or `self.story_generator = story_generator_instance`). This makes them accessible within the custom agent's core asynchronous execution logic (such as: `_run_async_impl` method).
- **Sub Agents List:** When initializing the `BaseAgent` using its `super()` constructor, you should pass a `sub_agents` list. This list tells the ADK framework about the agents that are part of this custom agent's immediate hierarchy. It's important for framework features like lifecycle management, introspection, and potentially future routing capabilities, even if you core execution logic (`_run_async_impl`) calls the agents directly via `self.xxx_agent`. Include the agents that your custom logic directly invokes at the top level.
- **State:** As mentioned, `ctx.session.state` is the standard way sub-agents (especially `LlmAgent`s using `output key`) communicate results back to the orchestrator and how the orchestrator passes necessary inputs down.

Design Pattern Example:

StoryFlowAgent

Let's illustrate the power of custom agents with an example pattern:

A multi-stage content generation workflow with conditional logic.

Goal: Create a system that generates a story, iteratively refines it through critique and revision, performs final checks, and crucially, *regenerates the story if the final tone check fails*.

Why Custom?

The core requirement driving the need for a custom agent here is the **conditional regeneration based on the tone check**. Standard workflow agents don't have built-in conditional branching based on the outcome of a sub-agent's task.

We need custom logic (if tone == "negative": ...) within the orchestrator.

Part 1: Simplified custom agent Initialization

We define the `StoryFlowAgent` inheriting from `BaseAgent`. In `__init__`, we store the necessary sub-agents (passed in) as instance attributes and tell the `BaseAgent` framework about the top-level agents this custom agent will directly orchestrate.

[part1_storyflow.py](#)

Part 2: Defining the Custom Execution Logic

This method orchestrates the sub-agents using standard Python async/await and control flow.

[part2_storyflow.py](#)

Explanation of Logic:

The initial `story_generator` runs. Its output is expected to be in `ctx.session.state["current_story"]`.

The `loop_agent` runs, which internally calls the `critic` and `reviser` sequentially for `max_iterations` times.

They read/write `current_story` and `criticism` from/to the state.

The `sequential_agent` runs, calling `grammar_check` then `tone_check`, reading `current_story` and writing `grammar_suggestions` and `tone_check_result` to the state.

Custom Part: The `if` statement checks the `tone_check_result` from the state. If it's "negative", the `story_generator` is called *again*, overwriting the `current_story` in the state. Otherwise, the flow ends.

Part 3: Defining the LLM Sub-Agents

These are standard `LlmAgent` definitions, responsible for specific tasks.

Their `output key` parameter is crucial for placing results into the `session.state` where other agents or the custom orchestrator can access them.

[part3_storyflow.py](#)

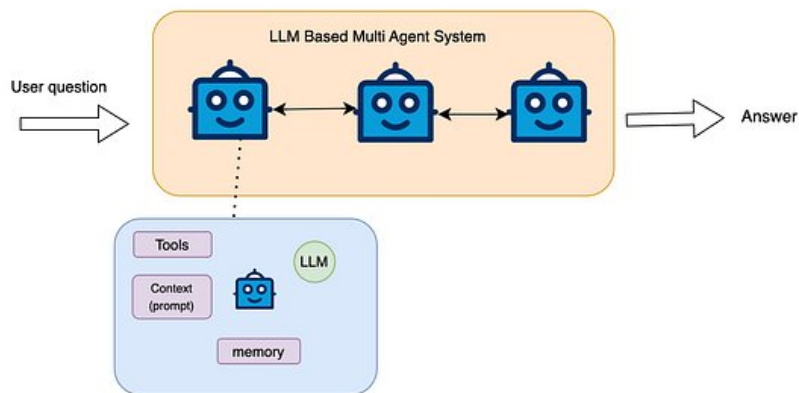
Part 4: Instantiating and Running the custom agent

Finally, you instantiate your `StoryFlowAgent` and use the `Runner` as usual.

[part4_storyflow.py](#)

[full_storyflowagent.py](#)

Multi-Agent Systems in ADK



Multi-Agent Systems in ADK

- **Why Not Just One Agent?**
 - A single, monolithic agent can be **hard to build, maintain, and reason about** as complexity grows.
- **What Is a Multi-Agent System (MAS)?**
 - An application made of **multiple BaseAgent instances** working together.
 - Agents may form a **hierarchy**, collaborating or coordinating to achieve a bigger goal.
- **Key Benefits**
 - **Modularity** → Break down into smaller, focused agents.
 - **Specialization** → Each agent can handle a specific task well.
 - **Reusability** → Reuse agents across projects or workflows.
 - **Maintainability** → Easier to update or replace individual agents.
 - **Structured Control** → Use workflow agents for predictable orchestration.
- **Types of Agents You Can Combine**
 1. **LLM Agents** → Language-driven reasoning & decision-making.
 2. **Workflow Agents** → Control flow (Sequential, Parallel, Loop).
 3. **Custom Agents** → Specialized logic or external integrations.

⚡ **Key Insight:** Multi-Agent Systems let you **mix and match agent types** to build applications that are both **intelligent and well-structured**.

1. ADK Primitives for Agent Composition

ADK provides core building blocks—primitives—that enable you to structure and manage interactions within your multi-agent system.

Agent Hierarchy (Parent & Sub-Agents)

- **Foundation:**
 - Multi-agent systems are structured as a **parent** → **child tree** using `BaseAgent`.
- **How to Establish Hierarchy:**
 - Pass a list of agents to the `sub_agents` argument when creating a parent agent.
 - ADK automatically sets each child's `parent_agent` attribute.
- **Single Parent Rule:**
 - An agent can belong to **only one parent**.
 - Adding the same agent to another parent → raises `ValueError`.
- **Why Hierarchy Matters:**
 - Defines the **scope for Workflow Agents** (who controls whom).
 - Determines possible **delegation targets** for LLM-driven decision-making.
 - You can navigate the tree:
 - `agent.parent_agent` → move upward.
 - `agent.find_agent(name)` → find descendants by name.

```
# Conceptual Example: Defining Hierarchy
from google.adk.agents import LlmAgent, BaseAgent

# Define individual agents
greeter = LlmAgent(name="Greeter", model="gemini-2.0-flash")
task_doer = BaseAgent(name="TaskExecutor") # Custom non-LLM agent

# Create parent agent and assign children via sub_agents
coordinator = LlmAgent(
    name="Coordinator",
    model="gemini-2.0-flash",
    description="I coordinate greetings and tasks.",
    sub_agents=[ # Assign sub_agents here
        greeter,
        task_doer
    ]
)

# Framework automatically sets:
# assert greeter.parent_agent == coordinator
# assert task_doer.parent_agent == coordinator
```

⚡ **Key Insight:** Hierarchy gives structure to multi-agent systems—ensuring **clear control, valid delegation paths, and predictable orchestration**.

Workflow Agents as Orchestrators

- What They Do
 - Special agents (derived from `BaseAgent`) that **don't perform tasks directly**.
 - Instead, they **orchestrate sub-agents** and control how they run.
- Example: `SequentialAgent`
 - Runs sub-agents **one after another** in the listed order.
 - Shares the same **InvocationContext** across steps.
 - This makes it easy for agents to **pass results through shared state**.

⚡ **Key Insight:** Workflow Agents act as **directors**, ensuring sub-agents execute in the **right order** with **shared context**, even though they don't do the actual work themselves.

Conceptual Example: Sequential Pipeline

```
from google.adk.agents import SequentialAgent, LlmAgent
```

```
step1 = LlmAgent(name="Step1_Fetch", output_key="data")
```

Saves output to state['data']

```
step2 = LlmAgent(name="Step2_Process", instruction="Process data from {data}.")
```

```
pipeline = SequentialAgent(name="MyPipeline", sub_agents=[step1, step2])
```

When pipeline runs, Step2 can access the state['data'] set by Step1.

Interaction & Communication – Shared Session State

- **What It Is**

- The **simplest way** for agents in the same invocation (same `Session` via `InvocationContext`) to exchange data.

- **How It Works**

- One agent writes to state:

```
python  
  
context.state["data_key"] = processed_data
```

- Another agent later reads from state:

```
python  
  
data = context.state.get("data_key")
```

- All state changes are tracked using **CallbackContext**.

- **Convenience**

- `output_key` property on an `LlmAgent` automatically saves its final response/output to a state key

- **Nature of Communication**

- **Asynchronous & Passive** → No direct "messaging," just reading/writing to shared state.
- Works well for:
 - **SequentialAgent** pipelines (passing data step by step).
 - **LoopAgent** iterations (carrying forward counters, flags, or results).

⚡ **Key Insight:** Shared `session.state` acts like a **common memory board** where agents can drop results for others to pick up later.

ParallelAgent as Orchestrator

- **Execution**
 - Runs all **sub-agents in parallel**.
 - Events from different agents may be **interleaved** during execution.
- **Context Handling**
 - Creates a unique **InvocationContext.branch** for each child agent.
 - Example: `ParentBranch.ChildName` → provides a **separate contextual path**.
 - Useful for keeping histories **isolated** in some memory setups.
- **State Management**
 - All sub-agents share the **same** `session.state`.
 - Can **read initial state** and **write results**.
 - ⚠️ Must use **unique keys** to prevent race conditions.

⚡ **Key Insight:** ParallelAgent allows **concurrent execution with shared state**, but requires **careful key management** to avoid conflicts.

Conceptual Example: Parallel Execution

```
from google.adk.agents import ParallelAgent, LlmAgent
```

```
fetch_weather = LlmAgent(name="WeatherFetcher", output_key="weather")
```

```
fetch_news = LlmAgent(name="NewsFetcher", output_key="news")
```

```
gatherer = ParallelAgent(name="InfoGatherer", sub_agents=[fetch_weather, fetch_news])
```

```
# When gatherer runs, WeatherFetcher and NewsFetcher run concurrently.
```

```
# A subsequent agent could read state['weather'] and state['news'].
```


LoopAgent as Orchestrator

- **Execution**
 - Runs its **sub-agents sequentially**.
 - Repeats in a **loop** until a stop condition is met.
- **Termination Conditions**
 - `max_iterations` → Loop ends after the set number of cycles.
 - **Escalation Event** → If a sub-agent returns an event with `escalate=True`, the loop stops immediately.
- **Context & State**
 - Uses the **same InvocationContext** across all iterations.
 - This allows **state changes** (like counters, flags, progress markers) to persist between loops.

⚡ **Key Insight:** LoopAgent is ideal for **iterative refinement**—it keeps running until either a condition tells it to stop or a maximum limit is reached.

Conceptual Example: Loop with Condition

```
from google.adk.agents import LoopAgent, LlmAgent, BaseAgent
from google.adk.events import Event, EventActions
from google.adk.agents.invocation_context import InvocationContext
from typing import AsyncGenerator
```

```
class CheckCondition(BaseAgent):                                # Custom agent to check state
    async def _run_async_impl(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]:
        status = ctx.session.state.get("status", "pending")
        is_done = (status == "completed")
        yield Event(author=self.name, actions=EventActions(escalate=is_done))    # Escalate if done

process_step = LlmAgent(name="ProcessingStep")                # Agent that might update state['status']

poller = LoopAgent(
    name="StatusPoller",
    max_iterations=10,
    sub_agents=[process_step, CheckCondition(name="Checker")]
)
# When poller runs, it executes process_step then Checker repeatedly
# until Checker escalates (state['status'] == 'completed') or 10 iterations pass.
```

Conceptual Example: Using output_key and reading state

```
from google.adk.agents import LlmAgent, SequentialAgent
```

```
agent_A = LlmAgent(name="AgentA", instruction="Find the capital of France.", output_key="capital_city")
```

```
agent_B = LlmAgent(name="AgentB", instruction="Tell me about the city stored in {capital_city}.")
```

```
pipeline = SequentialAgent(name="CityInfo", sub_agents=[agent_A, agent_B])
```

AgentA runs, saves "Paris" to state['capital_city'].

AgentB runs, its instruction processor reads state['capital_city'] to get "Paris".

Interaction & Communication – LLM-Driven Delegation (Agent Transfer)

- **What It Is**
 - Lets an **LlmAgent** dynamically **hand off a task** to another agent in the hierarchy.
 - Enables **flexible, intelligent routing** of tasks.
- **How It Works**
 - The LLM generates a call like:

```
python

transfer_to_agent(agent_name="target_agent_name")
```
 - **AutoFlow** intercepts this call and:
 - Uses `root_agent.find_agent()` to locate the target.
 - Updates the **InvocationContext** to transfer execution.
- **Requirements**
 - The **calling agent** must have clear instructions on **when to transfer**.
 - The **target agents** need distinct **descriptions**, so the LLM can pick the right one.
 - Transfer scope (parent, sub-agent, siblings) can be configured per agent.
- **Nature of Communication**
 - **Dynamic & Flexible** → Decisions depend on LLM interpretation.
 - Useful when the **next best agent** can't be determined in advance.

⚡ **Key Insight:** LLM-driven delegation turns your system into a **smart router**, where agents can “decide” or the fly who should handle the next step.

Conceptual Setup: LLM Transfer

```
from google.adk.agents import LlmAgent
```

```
booking_agent = LlmAgent(name="Booker", description="Handles flight and hotel bookings.")
```

```
info_agent = LlmAgent(name="Info", description="Provides general information and answers questions.")
```

```
coordinator = LlmAgent(
```

```
    name="Coordinator",
```

```
    model="gemini-2.0-flash",
```

```
    instruction="You are an assistant. Delegate booking tasks to Booker and info requests to Info.",
```

```
    description="Main coordinator.",
```

```
    # AutoFlow is typically used implicitly here
```

```
    sub_agents=[booking_agent, info_agent]
```

```
)
```

```
# If coordinator receives "Book a flight", its LLM should generate:
```

```
# FunctionCall(name='transfer_to_agent', args={'agent_name': 'Booker'})
```

```
# ADK framework then routes execution to booking_agent.
```

Interaction & Communication – Explicit Invocation (AgentTool)

- What It Is
 - Lets an **LlmAgent** call another agent directly, treating it like a **function/tool**.
- How It Works
 - Wrap the target agent in `AgentTool`.
 - Add it to the parent LlmAgent's `tools` list.
 - The LLM sees a generated **function declaration** for this tool.
 - When the LLM calls it → `AgentTool.run_async` runs:
 - Executes the target agent.
 - Captures its response.
 - Updates parent's **state/artifacts** with the results.
 - Returns the response as the tool's output.
- Nature of Communication
 - **Synchronous** → Runs inside the parent's flow.
 - **Explicit & Controlled** → The parent explicitly invokes another agent.
 - Behaves like **any other tool call**.

[AgentTool.py](#)

⚡ **Key Insight:** With `AgentTool`, an agent can be used as a **callable component**, making it easy to compose multi-agent workflows like function calls inside a program.

2. Common Multi-Agent Patterns using ADK Primitives

Multi-Agent Collaboration Pattern: Coordinator/Dispatcher

- **Structure**
 - A central LlmAgent (Coordinator) manages multiple **specialist sub-agents**.
- **Goal**
 - **Route requests** to the right specialist agent based on the task.
- **ADK Primitives Used**
 1. **Hierarchy** → Coordinator lists specialists in its `sub_agents`.
 2. **Interaction Methods:**
 - **LLM-Driven Delegation** → Coordinator dynamically transfers tasks to the correct specialist.
 - Needs clear **descriptions** on sub-agents.
 - Requires good **instructions** for Coordinator.
 - **Explicit Invocation (AgentTool)** → Coordinator calls specialists directly as tools.

⚡ **Key Insight:** The Coordinator/Dispatcher pattern turns one agent into a **smart router**, directing work to the right specialist for efficiency and clarity.

`# Conceptual Code: Coordinator using LLM Transfer`

```
from google.adk.agents import LlmAgent
```

```
billing_agent = LlmAgent(name="Billing", description="Handles billing inquiries.")
```

```
support_agent = LlmAgent(name="Support", description="Handles technical support requests.")
```

```
coordinator = LlmAgent(
```

```
    name="HelpDeskCoordinator",
```

```
    model="gemini-2.0-flash",
```

```
    instruction="Route user requests: Use Billing agent for payment issues, Support agent for technical problems.",
```

```
    description="Main help desk router.",
```

```
    # allow_transfer=True is often implicit with sub_agents in AutoFlow
```

```
    sub_agents=[billing_agent, support_agent]
```

```
)
```

```
# User asks "My payment failed" -> Coordinator's LLM should call transfer_to_agent(agent_name='Billing')
```

```
# User asks "I can't log in" -> Coordinator's LLM should call transfer_to_agent(agent_name='Support')
```

Multi-Agent Collaboration Pattern: Sequential Pipeline

- **Structure**
 - A **SequentialAgent** contains multiple **sub-agents** executed in a **strict, fixed order**.
- **Goal**
 - Build a **multi-step process** where each step's **output feeds into the next**.
- **ADK Primitives Used**
 1. **Workflow** → `SequentialAgent` enforces the execution order.
 2. **Communication** → Uses **Shared Session State**:
 - Early agents **write results** (often via `output_key`).
 - Later agents **read results** from `context.state`.

⚡ **Key Insight:** The Sequential Pipeline pattern is ideal for **step-by-step workflows** like data processing, code pipelines, or document refinement, where each stage depends on the previous one.

Conceptual Code: Sequential Data Pipeline

```
from google.adk.agents import SequentialAgent, LlmAgent
```

```
validator = LlmAgent(name="ValidateInput", instruction="Validate the input.", output_key="validation_status")
```

```
processor = LlmAgent(name="ProcessData", instruction="Process data if {validation_status} is 'valid'.", output_key="result")
```

```
reporter = LlmAgent(name="ReportResult", instruction="Report the result from {result}.")
```

```
data_pipeline = SequentialAgent(  
    name="DataPipeline",  
    sub_agents=[validator, processor, reporter]  
)
```

```
# validator runs -> saves to state['validation_status']
```

```
# processor runs -> reads state['validation_status'], saves to state['result']
```

```
# reporter runs -> reads state['result']
```

Multi-Agent Collaboration Pattern: Parallel Fan-Out/Gather

- **Structure**
 - A **ParallelAgent** runs multiple sub-agents **at the same time** (Fan-Out).
 - Typically followed by a **SequentialAgent** step to **aggregate results** (Gather).
- **Goal**
 - Perform **independent tasks concurrently** to save time.
 - Combine outputs afterward for a **unified result**.
- **ADK Primitives Used**
 1. **Workflow** →
 - **ParallelAgent** → concurrent execution (Fan-Out).
 - **SequentialAgent** → aggregation stage (Gather).
 2. **Communication** →
 - Each sub-agent writes its results to a **unique key** in `Shared Session State`.
 - The Gather agent then **reads all keys** to merge outputs.

⚡ **Key Insight:** The Parallel Fan-Out/Gather pattern is best when tasks are **independent but need to be merged later**—like querying multiple data sources or running simultaneous analyses before combining insights.

Conceptual Code: Parallel Information Gathering

```
from google.adk.agents import SequentialAgent, ParallelAgent, LlmAgent
```

```
fetch_api1 = LlmAgent(name="API1Fetcher", instruction="Fetch data from API 1.", output_key="api1_data")
```

```
fetch_api2 = LlmAgent(name="API2Fetcher", instruction="Fetch data from API 2.", output_key="api2_data")
```

```
gather_concurrently = ParallelAgent(  
    name="ConcurrentFetch",  
    sub_agents=[fetch_api1, fetch_api2]  
)
```

```
synthesizer = LlmAgent(  
    name="Synthesizer",  
    instruction="Combine results from {api1_data} and {api2_data}."  
)
```

```
overall_workflow = SequentialAgent(  
    name="FetchAndSynthesize",  
    sub_agents=[gather_concurrently, synthesizer] # Run parallel fetch, then synthesize  
)
```

fetch_api1 and fetch_api2 run concurrently, saving to state.

synthesizer runs afterwards, reading state['api1_data'] and state['api2_data'].

Multi-Agent Collaboration Pattern: Hierarchical Task Decomposition

- **Structure**
 - A **multi-level tree** of agents.
 - Higher-level agents **decompose complex goals** into smaller sub-tasks.
 - Lower-level agents handle the **execution** of those tasks.
- **Goal**
 - Tackle **complex problems** by breaking them down into **simpler, manageable steps**.
 - Results flow back **up the hierarchy** for integration.
- **ADK Primitives Used**
 1. **Hierarchy** → Multi-level `parent_agent` / `sub_agents` relationships.
 2. **Interaction** →
 - **LLM-Driven Delegation** → Parent dynamically routes sub-tasks.
 - **Explicit Invocation (AgentTool)** → Parent directly calls sub-agents.
 3. **Result Flow** → Sub-agent outputs return upward (via tool results or `session.state`).

⚡ **Key Insight:** Hierarchical Task Decomposition is powerful for **large, complex objectives**—it mirrors human problem-solving by **breaking down big goals into smaller, solvable parts**.

Conceptual Code: Hierarchical Research Task

```
from google.adk.agents import LlmAgent
from google.adk.tools import agent_tool
```

Low-level tool-like agents

```
web_searcher = LlmAgent(name="WebSearch", description="Performs web searches for facts.")
summarizer = LlmAgent(name="Summarizer", description="Summarizes text.")
```

Mid-level agent combining tools

```
research_assistant = LlmAgent(
    name="ResearchAssistant",
    model="gemini-2.0-flash",
    description="Finds and summarizes information on a topic.",
    tools=[agent_tool.AgentTool(agent=web_searcher), agent_tool.AgentTool(agent=summarizer)]
)
```

High-level agent delegating research

```
report_writer = LlmAgent(
    name="ReportWriter",
    model="gemini-2.0-flash",
    instruction="Write a report on topic X. Use the ResearchAssistant to gather information.",
    tools=[agent_tool.AgentTool(agent=research_assistant)]
)
```

Alternatively, could use LLM Transfer if research_assistant is a sub_agent

)

User interacts with ReportWriter.

ReportWriter calls ResearchAssistant tool.

ResearchAssistant calls WebSearch and Summarizer tools.

Results flow back up.

Multi-Agent Collaboration Pattern: Review/Critique (Generator–Critic)

- **Structure**
 - Two agents inside a **SequentialAgent**:
 1. **Generator** → produces content/output.
 2. **Critic/Reviewer** → evaluates and critiques that output.
- **Goal**
 - Enhance **quality, accuracy, or validity** by adding a review step after generation.
- **ADK Primitives Used**
 1. **Workflow** → `SequentialAgent` enforces order: generate → then review.
 2. **Communication** →
 - Generator saves output via `output_key` in **Shared Session State**.
 - Reviewer reads from that state key, and may save feedback to another key for follow-up steps.

⚡ **Key Insight:** The Generator–Critic pattern is a lightweight way to add **feedback loops** for better quality control—ideal for writing, coding, or analysis workflows.

Conceptual Code: Generator-Critic

```
from google.adk.agents import SequentialAgent, LlmAgent
```

```
generator = LlmAgent(  
    name="DraftWriter",  
    instruction="Write a short paragraph about subject X.",  
    output_key="draft_text"  
)
```

```
reviewer = LlmAgent(  
    name="FactChecker",  
    instruction="Review the text in {draft_text} for factual accuracy. Output 'valid' or 'invalid' with reasons.",  
    output_key="review_status"  
)
```

Optional: Further steps based on review_status

```
review_pipeline = SequentialAgent(  
    name="WriteAndReview",  
    sub_agents=[generator, reviewer]  
)
```

generator runs -> saves draft to state['draft_text']

reviewer runs -> reads state['draft_text'], saves status to state['review_status']

Multi-Agent Collaboration Pattern: Iterative Refinement

- **Structure**
 - A **LoopAgent** wraps one or more agents that run **repeatedly**.
- **Goal**
 - **Progressively improve** a result (code, text, plan, etc.) stored in `session.state` until it meets a **quality threshold** or hits a **max iteration** limit.
- **ADK Primitives Used**
 1. **Workflow** → `LoopAgent` controls the repetition.
 2. **Communication** → **Shared Session State**:
 - Agents **read** the previous iteration's output.
 - Agents **write** the refined version back to state.
- **Termination**
 - `max_iterations` → stop after N cycles, or
 - **Quality Check Agent** → sets `escalate=True` in an event when the result is satisfactory, stopping the loop early.

⚡ **Key Insight:** Iterative Refinement is perfect when you want **steady, step-by-step improvement** with a **clear stop rule** (quality met or iteration cap).

Conceptual Code: Iterative Code Refinement

```
from google.adk.agents import LoopAgent, LlmAgent, BaseAgent
from google.adk.events import Event, EventActions
from google.adk.agents.invocation_context import InvocationContext
from typing import AsyncGenerator
```

Agent to generate/refine code based on state['current_code'] and state['requirements']

```
code_refiner = LlmAgent(
    name="CodeRefiner",
    instruction="Read state['current_code'] (if exists) and state['requirements']. Generate/refine Python code to meet requirements. Save to state['current_code'].",
    output_key="current_code" # Overwrites previous code in state
)
```

Agent to check if the code meets quality standards

```
quality_checker = LlmAgent(
    name="QualityChecker",
    instruction="Evaluate the code in state['current_code'] against state['requirements']. Output 'pass' or 'fail'.",
    output_key="quality_status"
)
```

Custom agent to check the status and escalate if 'pass'

```
class CheckStatusAndEscalate(BaseAgent):
    async def _run_async_impl(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]:
        status = ctx.session.state.get("quality_status", "fail")
        should_stop = (status == "pass")
        yield Event(author=self.name, actions=EventActions(escalate=should_stop))
```

```
refinement_loop = LoopAgent(
    name="CodeRefinementLoop",
    max_iterations=5,
    sub_agents=[code_refiner, quality_checker, CheckStatusAndEscalate(name="StopChecker")]
)
```

Loop runs: Refiner -> Checker -> StopChecker

State['current_code'] is updated each iteration.

Loop stops if QualityChecker outputs 'pass' (leading to StopChecker escalating) or after 5 iterations.

Multi-Agent Collaboration Pattern: Human-in-the-Loop

- **Structure**
 - Adds **human checkpoints** inside an agent workflow.
 - Humans can **review, approve, correct, or handle tasks** AI cannot perform.
- **Goal**
 - Ensure **human oversight** and handle cases where automation alone is insufficient.
- **ADK Primitives Used (Conceptual)**
 1. **Interaction** →
 - Use a **custom Tool** that pauses execution and requests input from an external system (UI, ticketing, etc.).
 - The tool returns the **human's response** back to the agent.
 2. **Workflow** →
 - Could use **LLM-Driven Delegation** (`transfer_to_agent`) to a conceptual *Human Agent*.
 - Or use the **custom tool** directly inside an `LlmAgent` .
 3. **State & Callbacks** →
 - Store task details in `session.state` for humans to see.
 - Use callbacks to manage request/response flow.
- **Note**
 - ADK does **not** provide a built-in "Human Agent."
 - This requires **custom integration** with external systems.

⚡ **Key Insight:** Human-in-the-Loop ensures workflows stay **safe, correct, and practical** by combining automation speed with **human judgment** where it matters.

Conceptual Code: Using a Tool for Human Approval

```
from google.adk.agents import LlmAgent, SequentialAgent
from google.adk.tools import FunctionTool
```

```
# --- Assume external_approval_tool exists ---
```

```
# This tool would:
```

```
# 1. Take details (e.g., request_id, amount, reason).
```

```
# 2. Send these details to a human review system (e.g., via API).
```

```
# 3. Poll or wait for the human response (approved/rejected).
```

```
# 4. Return the human's decision.
```

```
# async def external_approval_tool(amount: float, reason: str) -> str: ...
```

```
approval_tool = FunctionTool(func=external_approval_tool)
```

Agent that prepares the request

```
prepare_request = LlmAgent(
    name="PrepareApproval",
    instruction="Prepare the approval request details based on user input. Store amount and reason in state.",
    # ... likely sets state['approval_amount'] and state['approval_reason'] ...
)
```

Agent that calls the human approval tool

```
request_approval = LlmAgent(
    name="RequestHumanApproval",
    instruction="Use the external_approval_tool with amount from state['approval_amount'] and reason from state['approval_reason'].",
    tools=[approval_tool],
    output_key="human_decision"
)
```

Agent that proceeds based on human decision

```
process_decision = LlmAgent(
    name="ProcessDecision",
    instruction="Check {human_decision}. If 'approved', proceed. If 'rejected', inform user."
)
```

```
approval_workflow = SequentialAgent(
    name="HumanApprovalWorkflow",
    sub_agents=[prepare_request, request_approval, process_decision]
)
```

All these patterns provide starting points for structuring your multi-agent systems.

You can mix and match them as needed to create the most effective architecture for your specific application.

Build agents with Agent Config

Agent Config in ADK

- **What It Is**
 - A way to **define and run agents without writing code**.
 - Uses a **YAML configuration file** instead of Python/Java classes.
- **Purpose**
 - Makes it easier for **non-developers** to build workflows.
 - Quickly assemble agents with descriptions, instructions, and capabilities.

- **Basic Example**

yaml

```
name: assistant_agent
model: gemini-2.5-flash
description: A helper agent that can answer users' questions.
instruction: You are an agent to help answer users' various questions.
```

- **Capabilities**
 - Start simple (like the example above).
 - Scale to **complex agents** that include:
 - **Functions**
 - **Tools**
 - **Sub-Agents**
 - And more advanced workflow elements.

⚡ **Key Insight:** Agent Config files let you **assemble, configure, and run agents declaratively** in YAML—making ADK workflows accessible even to those without coding skills.

Setup

You need to install the Google Agent Development Kit libraries, and provide an access key for a generative AI model such as Gemini API. This section provides details on what you must install and configure before you can run agents with the Agent Config files.

Install ADK 📄

```
pip install google-adk
```

(Optional) Verify your installation:

```
pip show google-adk
```

```
adk --version
```

This command should show the ADK version you have installed.

ADK agentconfig

1. Google AI Studio API key (no GCP/ADC needed)
2. Vertex AI on Google Cloud (needs ADC or a service-account key)

0) Create the agent scaffold

```
bash

# (Ensure Python and the ADK CLI `adk` are installed according to the docs)
adk create --type=config my_agent
cd my_agent
```

This gives you:

- `root_agent.yaml` (Agent Config)
- `.env` (where you put credentials)

1) Mode A — Use Google AI Studio API key (simplest; avoids ADC)

Edit `my_agent/.env` :

```
# Use direct API key path (no Google Cloud project required)
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=YOUR_GOOGLE_AI_STUDIO_API_KEY
```

Edit `my_agent/root_agent.yaml` :

```
# yaml-language-server: $schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json
name: assistant_agent
model: gemini-2.5-flash
description: A helper agent that can answer users' questions.
instruction: You are an agent to help answer users' various questions.
```

2) Mode B — Use Vertex AI on Google Cloud (needs ADC)

Edit `my_agent/.env` :

```
env

# Use Vertex AI (server-side on GCP)
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=your-gcp-project-id
GOOGLE_CLOUD_LOCATION=us-central1
```

Now set up **Application Default Credentials (ADC)** in your environment:

Option B1: Use your user ADC (good for local dev)

```
bash

# Logs you in and writes ADC to ~/.config/gcloud/application_default_credentials.json
gcloud auth application-default login

# (Optional) set project so tools know it
gcloud config set project your-gcp-project-id

# Quick sanity check
gcloud auth application-default print-access-token
```

Option B2: Use a Service Account (good for CI/prod)

1. In Google Cloud Console → IAM & Admin → Service Accounts → Create.
2. Grant roles your agent needs (e.g., Vertex AI User).
3. Create a JSON key and save it locally.
4. Point ADC to it:
 - Linux/macOS:

```
bash

export GOOGLE_APPLICATION_CREDENTIALS="/abs/path/service-account.json"
```

- Windows PowerShell:

```
powershell

$env:GOOGLE_APPLICATION_CREDENTIALS="C:\abs\path\service-account.json"
```

If ADC is missing/misconfigured while `GOOGLE_GENAI_USE_VERTEXAI=1`, you'll get:

```
pgsql

google.auth.exceptions.DefaultCredentialsError: Your default credentials were not found
```

The fixes are exactly the steps above.

3) Run the agent

From inside `my_agent/`:

```
bash
```

 Copy code

```
# Most common patterns (CLI varies slightly by ADK version—use --help to confirm)
```

```
adk run --config root_agent.yaml
```

```
# or
```

```
adk serve --config root_agent.yaml
```

```
# or
```

```
adk run config root_agent.yaml
```

Tip: `adk --help`, `adk run --help`, or `adk serve --help` will show the exact subcommands in your installed version.

Minimal files (final view)

`.env` (pick one mode):

env

```
# Mode A (API key)
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=YOUR_GOOGLE_AI_STUDIO_API_KEY
```

env

```
# Mode B (Vertex AI)
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=your-gcp-project-id
GOOGLE_CLOUD_LOCATION=us-central1
# (and ensure ADC is configured via gcloud or service-account JSON)
```

`root_agent.yaml`:

yaml

```
# yaml-language-server: $schema=https://raw.githubusercontent.com/google/ai
name: assistant_agent
model: gemini-2.5-flash
description: A helper agent that can answer users' questions.
instruction: You are an agent to help answer users' various questions.
```

Using Different Models with ADK

ADK primarily uses two mechanisms for model integration:

1. **Direct String / Registry:** For models tightly integrated with Google Cloud (like Gemini models accessed via Google AI Studio or Vertex AI) or models hosted on Vertex AI endpoints. You typically provide the model name or endpoint resource string directly to the `LlmAgent`. ADK's internal registry resolves this string to the appropriate backend client, often utilizing the `google-genai` library.
2. **Wrapper Classes:** For broader compatibility, especially with models outside the Google ecosystem or those requiring specific client configurations (like models accessed via LiteLLM). You instantiate a specific wrapper class (e.g., `LiteLlm`) and pass this object as the `model` parameter to your `LlmAgent`.

Using Google Gemini Models

Google AI Studio

This is the simplest method and is recommended for getting started quickly.

- **Authentication Method:** API Key
- **Setup:**
 - a. **Get an API key:** Obtain your key from [Google AI Studio](#).
 - b. **Set environment variables:** Create a `.env` file (Python) or `.properties` (Java) in your project's root directory and add the following lines. ADK will automatically load this file.

```
export GOOGLE_API_KEY="YOUR_GOOGLE_API_KEY"  
export GOOGLE_GENAI_USE_VERTEXAI=FALSE
```



(or)

Pass these variables during the model initialization via the `Client` (see example below).

- **Models:** Find all available models on the [Google AI for Developers site](#).

Google Cloud Vertex AI

For scalable and production-oriented use cases, Vertex AI is the recommended platform. Gemini on Vertex AI supports enterprise-grade features, security, and compliance controls. Based on your development environment and usecase, *choose one of the below methods to authenticate*.

Pre-requisites: A Google Cloud Project with [Vertex AI enabled](#).

Method A: User Credentials (for Local Development)

1. **Install the gcloud CLI:** Follow the official [installation instructions](#).
2. **Log in using ADC:** This command opens a browser to authenticate your user account for local development.

<https://cloud.google.com/vertex-ai/generative-ai/docs/models>

```
gcloud auth application-default login
```



3. **Set environment variables:**

```
export GOOGLE_CLOUD_PROJECT="YOUR_PROJECT_ID"  
export GOOGLE_CLOUD_LOCATION="YOUR_VERTEX_AI_LOCATION" # e.g., us-central1
```



Explicitly tell the library to use Vertex AI:

```
export GOOGLE_GENAI_USE_VERTEXAI=TRUE
```



Method B: Vertex AI Express Mode

Vertex AI Express Mode offers a simplified, API-key-based setup for rapid prototyping.

1. **Sign up for Express Mode** to get your API key.
2. **Set environment variables:**

```
export GOOGLE_API_KEY="PASTE_YOUR_EXPRESS_MODE_API_KEY_HERE"  
export GOOGLE_GENAI_USE_VERTEXAI=TRUE
```

Method C: Service Account (for Production & Automation)

For deployed applications, a service account is the standard method.

1. **Create a Service Account** and grant it the Vertex AI User role.
2. **Provide credentials to your application:**
 - **On Google Cloud:** If you are running the agent in Cloud Run, GKE, VM or other Google Cloud services, the environment can automatically provide the service account credentials. You don't have to create a key file.
 - **Elsewhere:** Create a [service account key file](#) and point to it with an environment variable:

```
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/your/keyfile.json"
```

Instead of the key file, you can also authenticate the service account using Workload Identity. But this is outside the scope of this guide.

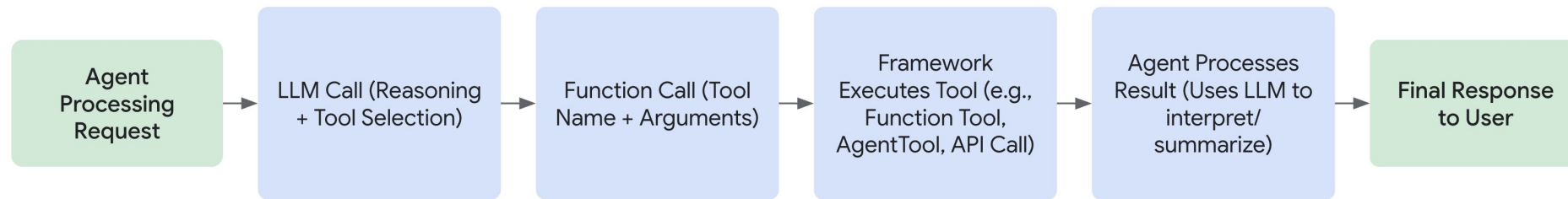
Tools

🔧 What is a Tool in ADK?

In the Agent Development Kit (ADK), a **Tool** is:

- **Definition:** A modular capability you “plug into” an agent.
- **Purpose:** Extend the agent beyond text-only reasoning so it can *take actions* and *interact with systems*.
- **Implementation:** Usually a function, class method, or sub-agent that exposes a task the agent can call.

Without tools, an agent is just a **language model**. With tools, it becomes a **capable agent** that can read/write data, call APIs, access databases, or control workflows.



Key Characteristics

1. Composable

Tools are modular. You can attach or detach them depending on what capabilities you want your agent to have.

2. Declarative

In an `Agent Config` YAML file, you describe the tools available. The ADK runtime loads them and makes them callable by the agent.

3. Bridging external systems

Tools typically provide an interface to something “outside” the model — like a database query, HTTP request, or sending an email.

4. LLM-Oriented API

Tools are designed so the language model can call them using natural reasoning, but under the hood, it's deterministic code.



Examples of Tools

1. Data Lookup Tool

- **Purpose:** Fetch data from a company knowledge base.
- **Implementation:** Python function:

```
python

def get_employee_email(employee_name: str) -> str:
    # Query HR database
    return "alice@example.com"
```

- **Agent Use:** "What's Alice's email?" → agent calls tool → tool returns answer.

2. External API Tool

- **Purpose:** Fetch weather data.
- **Implementation:**

```
python

import requests

def get_weather(city: str) -> str:
    resp = requests.get(f"https://api.weatherapi.com/{city}")
    return resp.json()["forecast"]
```

- **Agent Use:** "What's the weather in Delhi?" → agent calls `get_weather("Delhi")`.

3. Human-in-the-Loop Tool

- **Purpose:** Pause execution for human approval.
 - **Implementation:** Tool posts a Slack/Teams message and waits for user response.
 - **Agent Use:** "Should I approve this purchase order?" → agent asks tool → human responds → agent continues.
-

4. Another Agent as a Tool

- **Purpose:** Delegate tasks.
- **Implementation:** The "Finance Agent" is itself a tool the "Executive Agent" can call.
- **Agent Use:** "Summarize Q2 expenses" → root agent calls finance agent → result returned.

Tool Types in ADK 📌

ADK offers flexibility by supporting several types of tools:

1. **Function Tools:** Tools created by you, tailored to your specific application's needs.
 - **Functions/Methods:** Define standard synchronous functions or methods in your code (e.g., Python def).
 - **Agents-as-Tools:** Use another, potentially specialized, agent as a tool for a parent agent.
 - **Long Running Function Tools:** Support for tools that perform asynchronous operations or take significant time to complete.
2. **Built-in Tools:** Ready-to-use tools provided by the framework for common tasks. Examples: Google Search, Code Execution, Retrieval-Augmented Generation (RAG).
3. **Third-Party Tools:** Integrate tools seamlessly from popular external libraries. Examples: LangChain Tools, CrewAI Tools.

How Tools Show Up in Agent Config

Example `root_agent.yaml`:

yaml

 Copy code

```
name: assistant_agent
model: gemini-2.5-flash
description: A helper agent with extended capabilities.
instruction: You are an agent to help answer users' questions.

tools:
  - name: weather_tool
    description: Get current weather for a city
    function: my_tools.get_weather
  - name: employee_lookup
    description: Look up employee contact info
    function: hr_tools.get_employee_email
```

Here, the agent is not limited to text — it can now **call the** `weather_tool` or `employee_lookup` as needed.

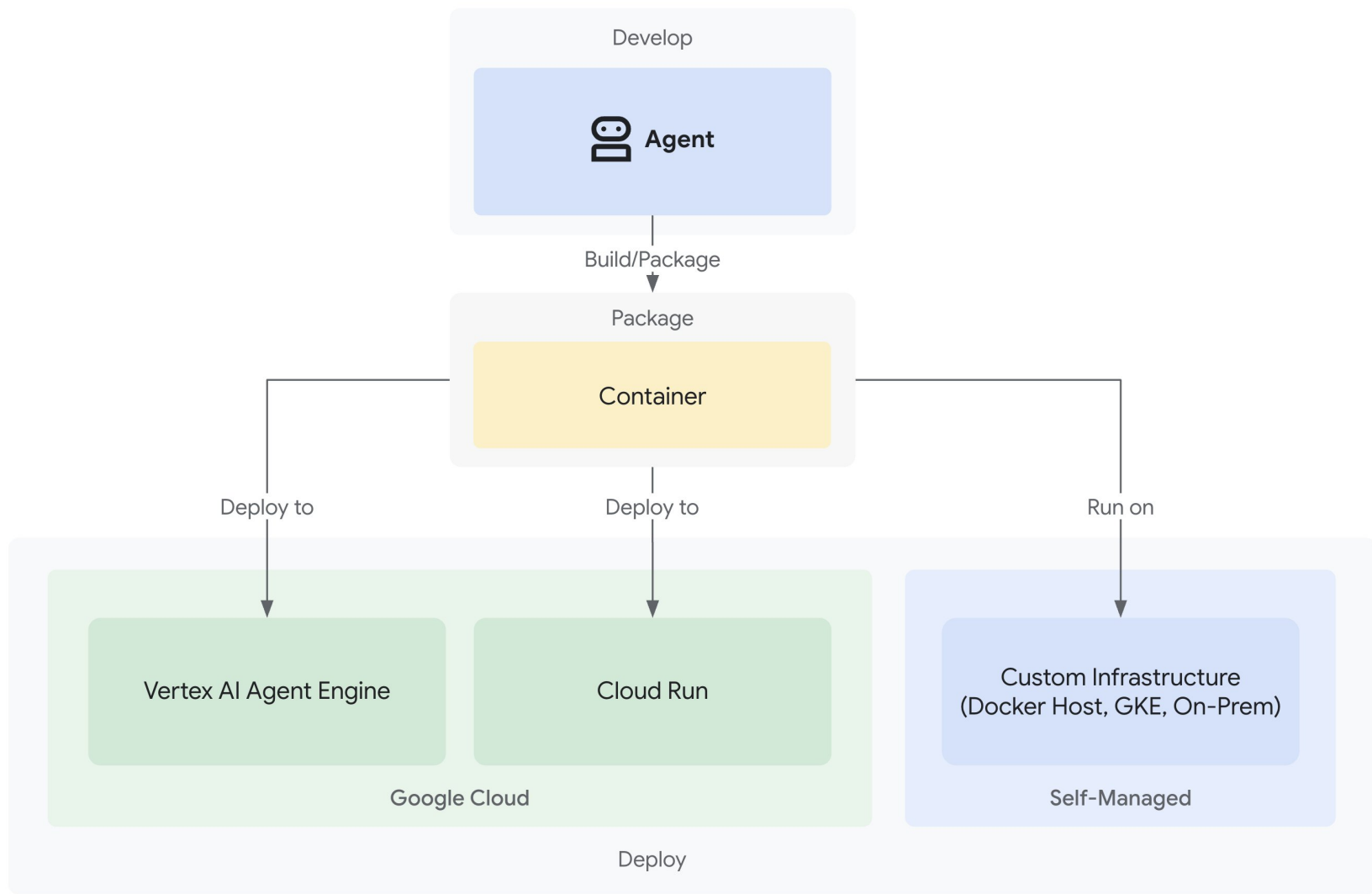
Why Tools Matter

- Agents without tools → only generate text.
- Agents with tools → can *act, query, fetch, update, control systems*.

This is the core leap from *LLM chatbot* → *Autonomous Agent*.

refere - [#tools.py](#)

Deploying Your Agent



References -

<https://google.github.io/adk-docs/>

<https://github.com/google/adk-python>

<https://github.com/a2aproject/a2a-samples/tree/main/demo>

<https://www.oreilly.com/radar/designing-collaborative-multi-agent-systems-with-the-a2a-protocol/>

<https://cloud.google.com/vertex-ai/generative-ai/docs>

<https://a2a-protocol.org/latest/>

<https://google.github.io/adk-docs/mcp/>